CHAPTER 2

Fundamentals of Probability

Probability is a field of mathematics that quantifies our uncertainty regarding events. For example, when rolling dice or flipping a coin, barring any irregularities in the dice or coin themselves, we are uncertain about the result to come. However, we can quantify our belief in each of the potential outcomes via probabilities. We say, for example, that on every coin toss the probability of the coin showing up heads is $\frac{1}{2}$. And on every dice roll, we say the probability of a die facing up with a five is $\frac{1}{6}$. These are the sorts of probabilities we talk about with ease in our daily lives, but how can we define and utilize them effectively? In this chapter we'll discuss the fundamentals of probability and how they connect to key concepts in deep learning.

Events and Probability

When running a trial such as rolling a dice or tossing a coin, we intuitively assign some belief to the trial's possible outcomes. In this section, we aim to formalize some of these concepts. In particular, we will begin by working in this *discrete* space, where discrete signifies a finite or countably infinite number of possibilities. Both rolling a dice and tossing a coin are in the discrete space—when rolling a fair dice there are six possible outcomes and when tossing a fair coin there are two. We term the entire set of possibilities for an experiment the *sample space*. For example, the numbers one through six would make up the sample space for rolling a fair dice. We can define *events* as subsets of the sample space. The event of rolling at least a three corresponds with the dice facing up any number in the subset of three, four, five, and six in the sample space defined previously. A set of probabilities that sum to one over all outcomes in the sample space is termed a *probability distribution* over that sample space, and these distributions will be the main focus of our discussion.

In general, we won't worry too much about where exactly these probabilities come from, as that requires a much more rigorous and thorough examination beyond the scope of this text. However, we will give some intuition about the different interpretations. At a high level, the *frequentist* view sees the probability of an outcome as arising from its frequency over a long-run experiment. In the case of fair dice, this view claims we can say the probability of any side of the dice showing up on a given roll is $\frac{1}{6}$, since performing a large number of rolls and counting up the occurrences of each side will give us an estimate that is roughly this fraction. As the number of rolls in the experiment grows, we see that this estimate gets closer and closer to the limit $\frac{1}{6}$, the outcome's probability.

On the other hand, the *Bayesian* view of probability is based more on quantifying our prior belief in hypotheses and how we update our beliefs in light of new data. For a fair dice, the Bayesian view would claim there is no prior information, both from the dice's structure and the rolling process, that would suggest any side of the dice as being more likely to turn up than any other side. Thus, we would say each outcome has probability $\frac{1}{6}$, our prior belief. The set of probabilities, in this case all being $\frac{1}{6}$, associated with each outcome is termed our *prior.* As we see new data, the Bayesian view gives us a methodology to update our prior accordingly, where we term this new belief our *posterior.* This Bayesian view is sometimes directly applied to neural network training, where we first assume that each weight in the network has some prior associated with it. As we train the network, we update the prior associated with each weight accordingly to better fit the data we see. At the end of the training, we are left with a posterior distribution associated with each weight.

We will assume throughout this chapter that the probabilities associated with any outcome have been determined via reasonable methods, and focus on how we can manipulate these probabilities for use in our analyses. We start with the four tenets of probability, specifically in the discrete space:

1. The sum of probabilities for all possible outcomes in a sample space must be equal to one. In other words, the probability distribution over the sample space must sum to one. This should make sense intuitively, since the set of all outcomes in the sample space must represent the entire set of possibilities. The probability distribution not summing to one would imply the existence of possibilities not accounted for, which is contradictory. Mathematically, we say that for any valid probability distribution, $\sum_o P(o) = 1$, where o represents an outcome.
2. Let E_1 be an event, and recall that we define an event as a subset of possible outcomes. We call E_1^c the *complement* of E_1, or all possible outcomes in the sample space that are not in E_1. The second tenet of probability is that $P(E_1) = 1 - P(E_1^c)$. This is just an application of the first tenet—if this were not true, it would clearly contradict the first tenet. In Figure 2-1, we see an example

of this, where S represents the entire space of outcomes, and the event and its complement together form the entirety of S.

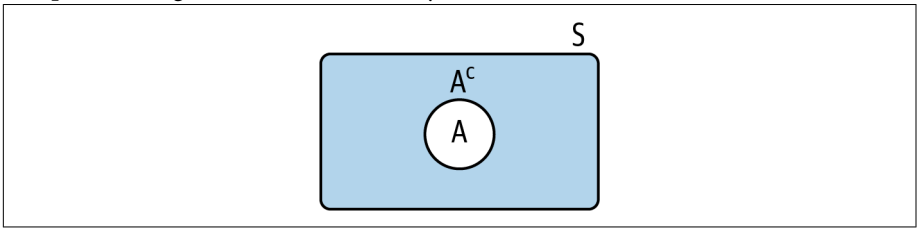


Figure 2-1. Event A and its complement interact to form the entire set of possibilities, S. The complement simply defines all the possibilities not originally in A.

3. Let E_1 and E_2 be two events, where E_1 is a subset (not necessarily strict) of E_2. The third tenet is that $P(E_1) \leq P(E_2)$. This, again, shouldn't be too surprising—the second event has at least as many outcomes as the first event, and all the outcomes the first event has since the second is a superset of the first. If this tenet were not true, that would imply the existence of outcomes with negative probability, which is impossible from our definitions.
4. The fourth and last tenet of probability is the principle of inclusion and exclusion, which states that $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. For those not familiar with this terminology, the $\cup$ denotes the *union* of the two events, a set operation that takes the two events and returns an event that contains all elements from the two original sets. The $\cap$, or *intersection,* is a set operation that returns an event that contains all elements belonging to both of the two original sets. The idea behind the equality presented is that by just naively summing the probabilities of A and B, we double-count the elements that belong to both sets. Thus, to accurately obtain the probability of the union, we must subtract the probability of the intersection. In Figure 2-2, we show two events and what their intersection would look like physically, while the union is all the outcomes in the combined area of the events.

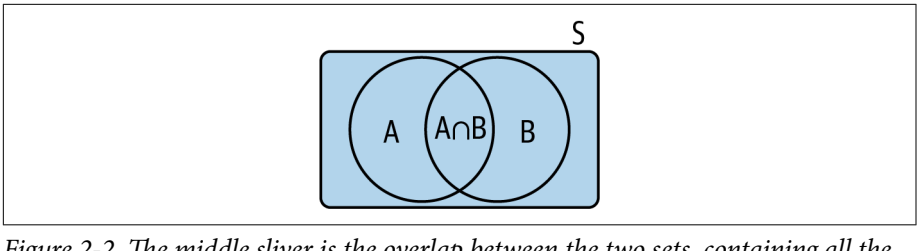


Figure 2-2. The middle sliver is the overlap between the two sets, containing all the outcomes that are in both sets. The union is all the events in the combined area of the two circles; if we were to add their probabilities naively, we would double-count all the outcomes in the middle sliver.

These tenets of probability find their way into everything that has to do with the field. For example, in deep learning, most of our problems fall into one of two categories: *regression* and *classification*. In the latter, we train a neural model that can predict the likelihood that the input belongs to one of a discrete number of classes. The famous MNIST digits dataset, for example, provides us with pictures of digits and associated numerical labels in the range of 0 through 9. Our objective is to build a *classifier* that can take in this picture and return the most likely label as its guess. This is naturally formulated as a problem in probability—the classifier produces a probability distribution over the sample space, 0 through 9, for any given input and its best guess is the digit that is assigned the highest probability. How does this relate to our tenets? Since the classifier is producing a probability distribution, it must follow the tenets. For example, the probabilities associated with each digit must sum to one—a quick back-of-the-envelope check to ensure the model isn't buggy. In the next section, we cover probabilities where we are initially given relevant information that affects our beliefs and how to use that information.

Conditional Probability

Knowing information often changes our beliefs, and by consequence, our probabilities. Going back to our classic dice example, we may roll the dice thinking that it's fair, while in reality there's a hidden weight at the dice's core, making it more likely to land up a number greater than three. As we roll the dice, we of course start to notice this pattern, and our belief regarding the dice's fairness starts to shift. This is at the core of conditional probability itself. Instead of thinking simply about $P(biased)$ or $P(fair)$, we have to think about probabilities like $P(biased|information)$ instead. This quantity, which we term a *conditional probability,* is spoken as "the probability the dice is biased *given* the information we've seen."

How do we think about such probabilities intuitively? For starters, we must imagine that we are now in a different universe than the one we started in. The new universe is one that incorporates the information we've seen since the start of the experiment, e.g., our past dice rolls. Going back to our MNIST example, the probability distribution that the trained neural net produces is actually a conditional probability distribution. The probability that the input image is zero, for example, can be seen as *P(0|input).* In plain English, we want to find the probability of a zero given all of the pixels that make up the specific input image we fed into our neural net. Our new universe is the universe in which the input pixels have taken on this specific configuration of values. This is distinct from simply looking at *P(0),* the probability of returning a zero, which we can think about in terms of prior belief. Without any knowledge of the input pixel configuration, we'd have no reason to believe that the possibility of returning a zero is any more or less likely than that of any other digit.

Sometimes, seeing certain information does not change our probabilities—we call this property *independence*. For example, Tom Brady may have thrown a touchdown pass after the third roll of our experiment, but incorporating that information into our new universe should (hopefully!) have no impact on the likelihood of the dice being biased. We state this independence property as *P(biased|Tom Brady throws a touchdown pass) = P(biased)*. Note that any two events E_1 and E_2 that satisfy this property are independent. Perhaps slightly more counterintuitively, if it happens to be the case that all of our dice rolls so far don't numerically change our prior belief regarding the dice's fairness (maybe the dice rolls so far have shown up evenly across one through six and our initial prior belief was that the dice was fair), we'd still say that these events are independent. Finally, note that independence is symmetric: if $P(E_1 \mid E_2) = P(E_1)$, then it is also the case that $P(E_2 \mid E_1) = P(E_2)$.

In the previous section, we introduced intersection and union notation. It turns out that we can break down the intersection operation into a product of probabilities. We have the following equality: $P(E_1 \cap E_2) = P(E_1 \mid E_2) * P(E_2)$. Let's break down the intuition here. On the left side, we have the probability that both events E_1 and E_2 have occurred. On the right side, we have the same idea, but expressed slightly differently. In the universe where both events have occurred, one way to arrive in this universe is to first have E_2 occur, followed by E_1. Porting this intuition into mathematical terms, we must first find the probability that E_2 has occurred, followed by the probability that E_1 has occurred in the universe where E_2 has already occurred. How do we combine these two probabilities? Intuitively, it makes sense that we multiply them—we must have both events occur, the first unconditionally and the second in the universe where the first has already occurred. Note that the order of these events doesn't really matter, as both paths get us to the same universe. So, more completely, $P(E_1 \cap E_2) = P(E_1 \mid E_2) * P(E_2) = P(E_2 \mid E_1) * P(E_1)$.

However, some of these paths make much more physical sense than others. For example, if we think of E_1 as the event where someone contracts a disease, and E_2 as the event where the patient shows symptoms of the disease, the path in which the patient contracts the disease and then shows symptoms makes much more physical sense than the reverse.

In the case where the two events are independent, we have that $P(E_1 \cap E_2) = P(E_1 \mid E_2) * P(E_2) = P(E_1) * P(E_2)$. Hopefully this makes some intuitive sense. In the independence scenario, the fact that E_2 has occurred doesn't affect the chances of E_1 occurring; i.e., incorporating this information into the new universe doesn't affect the probability of the next event. In the next section, we cover random variables, which are relevant summaries of events and also have their own probability distributions.

Random Variables

Once again, let's consider the coin flipping experiment. If we flip a coin some finite number of times, natural questions start to arise. How many heads did we encounter during our experiment? How many tails? How many tails until the first head? Every outcome in such an experiment has an answer to each of the listed questions. If we flip a coin say, five times, and we receive the sequence TTHHT, we have seen two heads, three tails, and two tails until the first head.

We can think of a *random variable* as a map, or a function, from the sample space to another space, such as the integers in Figure 2-3. Such a function would take as input the sequence TTHHT and output one of the three answers listed depending on the question we ask. The value that the random variable takes on would be the output associated with result of the experiment. Although random variables are deterministic in that they map a given input to a single output, they are not deterministic in that they also have a distribution associated with their output space. This is due to the inherent randomness in the experiment—depending on the probability of the input outcome, its corresponding output may be more or less likely than other outputs.

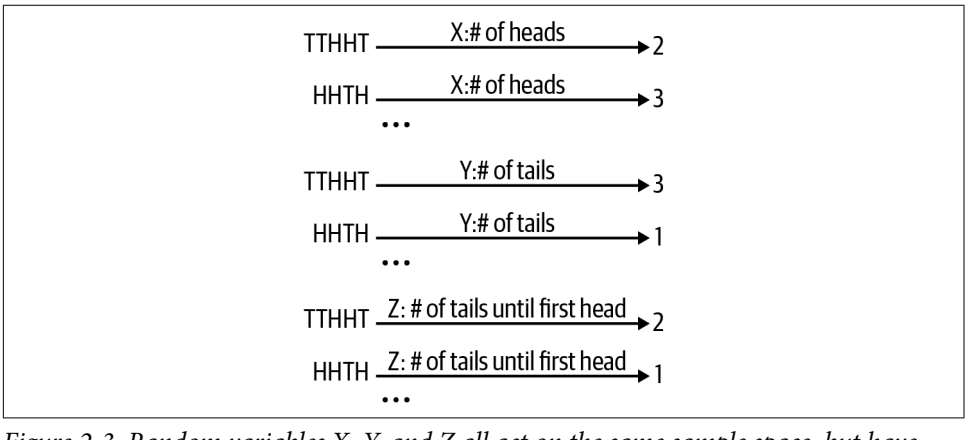


Figure 2-3. Random variables X, Y, and Z all act on the same sample space, but have varying outputs. It's important to keep in mind what you're measuring!



Note that multiple inputs could map to the same output. For example, *X(HHH)* = 3 in addition to *X(HHTH)* in Figure 2-3.

One easy way to begin is to just think of this map as an identity function—whatever we flip or roll, its map in the output space is exactly the same as the input. Encoding a heads as a one and a tails as a zero, we can define a random variable representing

the coin flip as whether the coin came up heads, i.e., *C(1) = 1,* where *C* is our random variable. In the dice scenario, the mapped output is the same as whatever we rolled, i.e., *D(5) = 5,* where *D* is our random variable.

Why should we care about random variables and their distributions? It turns out they play a vital role in deep learning and machine learning as a whole. For example, in Chapter 4, we will cover the concept of *dropout*, a technique for mitigating overfitting in neural networks. The idea of a dropout layer is that, during training, it independently and at random masks every neuron in the previous layer with some probability. This prevents the network from becoming overly dependent on specific connections or subnetworks. We can think of every neuron in the previous layer as representing a coin flip-type experiment. The only difference is that we set the probability of this experiment, rather than a fair coin having the default probability $\frac{1}{2}$ of showing up either side. Each neuron has a random variable *X* associated with it, with input one if the dropout layer decides to mask it and zero otherwise. *X* is an identity function from the input space to the output space, i.e., *X(1) = 1* and *X(0) = 0*.

Random variables, in general, need not be the identity map. Most functions you can think of are valid methods of mapping the input space to an output space where the random variable is defined. For example, if the input space were every possible length *n* sequence of coin flips, the function could be to count the number of heads in the sequence and square it. Some random variables can even be expressed as functions of other random variables, or a function of a function, as we will cover later. If we again consider the input space of every possible length *n* sequence of coin flips, the random variable counting the number of heads in the input sequence is the same as counting whether each individual coin flip turned up heads and taking a sum of all of those values. In mathematical terms, we say $X = \sum_{i=1}^{n} C_i$, where *X* is the random variable representing the total number of heads, and C_i is the binary random variable associated with the *i*th coin flip. Back to the dropout example, we can think of the random variable representing the total number of masked-out neurons as the sum of binary random variables representing each neuron.

In the future, when we want to refer to the event where the random variable takes on a specific value *c* (the domain being the output space we've been referring to, e.g., the number of heads in a sequence of coin flips), we will write this concisely as *X = c*. We denote the probability that the random variable takes on a specific value as *P(X = c),* for example. The probability that the random variable takes on any given value in the output space is just the sum of the probabilities of the inputs that map to it. This should make some intuitive sense, as this is basically the fourth tenet of probability where the intersection between any two events is the empty set since all the events we start from are individual, distinct inputs. Note that *P(X)* itself is also a probability distribution that follows all the basic tenets of probability described in the first section. In the next section, we consider statistics regarding random variables.

Expectation

As we discussed, a random variable is a map from input space to output space, where inputs are generated according to some probability distribution. The random variable can be thought of as a relevant summary of the input, and can take on many forms depending on the question we ask. Sometimes, it's useful to understand statistics regarding the random variable. For example, if we flip a coin eight times, how many heads do we expect to see on average? And, of course, we don't see the average number of heads all the time—how much does the number of heads we see tend to vary? The first quantity is what we call the random variable's *expectation*, and the second is the random variable's *variance*.

For a random variable X, we denote its expectation as $\mathbb{E}[X]$. We can think of this as the average value that X takes on, weighted by the probability of each of those outcomes. Mathematically, this is written as $\mathbb{E}[X] = \sum_o o * P(X = o)$. Note that if all outcomes o are equally likely, we get a simple average of all the outcomes. It makes sense to use the probability of the outcome as a weighting, since some outcomes are more likely than others, and the average value we observe will be skewed toward such outcomes. For a single fair coin flip, the expected number of heads would be $\sum_{o \in \{0,1\}} o * P(o) = 0 * 0.5 + 1 * 0.5 = 0.5$. In other words, we'd expect to see half of a head for any given fair coin flip. Of course, this makes no physical sense in that we could never possibly flip half of a head, but this gives you an idea of the proportions we'd expect to see over a long run experiment.

Returning to our example of length n sequences of coin flips, let's try to find the expected number of heads in such a sequence. We have $n + 1$ possible number of heads, and according to our formula, we'd need to find the probability of attaining each possible number to use as our weights. Mathematically, we'd need to compute $\sum_{x \in \{0,\ldots,n\}} x * P(X = x)$, where X is the random variable representing the total number of heads. However, as n gets larger and larger, performing this calculation starts to become more and more complicated.

Instead, let's denote X_i as the binary random variable for the ith coin flip and use the observation we made in the last section of being able to break up the total number of heads into a sum over heads/tails for all the individual coin flips. Since we know $X = X_1 + X_2 + \ldots + X_n$, we can also say that $\mathbb{E}[X] = \mathbb{E}[X_1 + X_2 + \ldots + X_n]$. How does making this substitution make our problem easier? We now introduce the concept of *linearity of expectation,* which states we can break up the right side into the sum $\mathbb{E}[X_1] + \mathbb{E}[X_2] + \ldots + \mathbb{E}[X_n]$. We know that the expected number of heads for each flip is 0.5, so the expected number of heads in a sequence of n flips is just $0.5*n$. This is much simpler than going down the previous route, as this approach's difficulty does not scale with the number of flips.

Let's go over the simplification we made in a bit more detail. Mathematically, if we have any two independent random variables A and B:

$$\mathbb{E}[A + B] = \Sigma_{a,b}(a + b) * P(A = a, B = b)$$
$$= \Sigma_{a,b}(a + b) * P(A = a) * P(B = b)$$
$$= \Sigma_{a,b} a * P(A = a) * P(B = b) + b * P(A = a) * P(B = b)$$
$$= \Sigma_{a,b} a * P(A = a) * P(B = b) + \Sigma_{a,b} b * P(A = a) * P(B = b)$$
$$= \Sigma_a a * P(A = a)\Sigma_b P(B = b) + \Sigma_b b * P(B = b)\Sigma_a P(A = a)$$
$$= \Sigma_a a * P(A = a) + \Sigma_b b * P(B = b)$$
$$= \mathbb{E}[A] + \mathbb{E}[B]$$



Note that we made the independence assumption we talked about earlier in the chapter here when we broke up the probability of the event $A = a$ and the event $B = b$ into a product of the two individual probabilities. The rest of the derivation doesn't require additional assumptions, so we recommend working through the algebra on your own. Although we won't show this for the dependent case, linearity of expectation also holds for dependent random variables.

Going back to the dropout example, the expectation of the total number of masked neurons can be broken up into a sum of expectations over each neuron. The expected number of masked neurons, similarly to the expected number of heads in a sequence of coin flips, is *p*n,* where *p* is the probability of being masked (and the expectation of each individual binary random variable representing a neuron) and *n* is the number of neurons.

As mentioned, we don't always see the expected number of occurrences of an event in every repetition of an experiment. In some cases, such as the expected number of heads in a single, fair coin flip from earlier, we never see it! Next, we will quantify the average deviation, or variance, from the expected value we see in repetitions of an experiment.

Variance

We define the variance, or Var(X), as $\mathbb{E}\left[(X - \mu)^2\right]$, where we let $\mu = \mathbb{E}[X]$. In plain English, this measure represents the average squared difference between the value X takes on and its expectation. Note that $(X - \mu)^2$ itself is also a random variable since it is a function of a function (X), which is still a function. Although we won't get into

too much detail about why we use this formula in particular, we encourage you to think about why we don't use a formula such as $\mathbb{E}[X - \mu]$ instead. To obtain a slightly simpler form for the variance, we can perform the following simplification:

$$\mathbb{E}\left[(X-\mu)^2\right] = \mathbb{E}\left[X^2 - 2\mu X + \mu^2\right]$$
$$= \mathbb{E}\left[X^2\right] - \mathbb{E}[2\mu X] + \mathbb{E}\left[\mu^2\right]$$
$$= \mathbb{E}\left[X^2\right] - 2\mu\mathbb{E}[X] + \mu^2$$
$$= \mathbb{E}\left[X^2\right] - 2\mathbb{E}[X]^2 + \mathbb{E}[X]^2$$
$$= \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2$$

Let's take a moment to go through each of these steps. In the first step, we fully express the random variable as all of its component terms via classic binomial expansion. In the second step, we perform linearity of expectation to break out the component terms into their own, individual expectations. In the third step, we note that μ, or $\mathbb{E}[X]$, and its square are both constants and thus can be pulled out of the surrounding expectation. They are constants since they are not a function of the value X takes on and are instead evaluated using the entire domain (the set of values X can take on). Constants can be seen as random variables that can take on only one value, which is the constant itself. Thus, their expectations, or the average value the random variable takes on, is the constant itself since we always see the constant. The final steps are algebraic manipulations that bring us to the simplified result. Let's use this formula to find the variance of the binary random variable representing a single neuron under dropout, and p is the probability of the neuron being masked out:

$$\mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2 = \Sigma_{x \in 0,1} x^2 * P(X = x) - \left(\Sigma_{x \in 0,1} x * P(X = x)\right)^2$$
$$= \Sigma_{x \in 0,1} x^2 * P(X = x) - p^2$$
$$= p - p^2$$
$$= p(1 - p)$$

These simplifications should make sense. We know from "Expectation" on page 24 that the expectation of the binary random variable representing a neuron is just p, and the rest is algebraic simplifications. We highly encourage you to work through these derivations on your own. As we start to think about the random variable representing the number of masked neurons in the entire layer, we naturally ask the question of whether there exists a similar linearity property for variance as there does for expectation. Unfortunately, the property does not hold in general:

$$
\begin{aligned}
Var(A+B) &= \mathbb{E}\left[(A+B)^2\right] - \mathbb{E}[A+B]^2 \\
&= \mathbb{E}\left[A^2 + 2*A*B + B^2\right] - (\mathbb{E}[A] + \mathbb{E}[B])^2 \\
&= \mathbb{E}\left[A^2\right] + 2\mathbb{E}[A*B] + \mathbb{E}\left[B^2\right] - \mathbb{E}[A]^2 - 2\mathbb{E}[A]\mathbb{E}[B] - \mathbb{E}[B]^2 \\
&= \mathbb{E}\left[A^2\right] - \mathbb{E}[A]^2 + \mathbb{E}\left[B^2\right] - \mathbb{E}[B]^2 + 2\mathbb{E}[A*B] - 2\mathbb{E}[A]\mathbb{E}[B] \\
&= Var(A) + Var(B) + 2(\mathbb{E}[A*B] - \mathbb{E}[A]\mathbb{E}[B]) \\
&= Var(A) + Var(B) + 2Cov(A,B)
\end{aligned}
$$

As we can see from the last line, the final term in the expression, which we call the *covariance* between the two random variables, ruins our hope for linearity. However, covariance is another key concept in probability—the intuition for covariance is that it measures the dependence between two random variables. As one random variable more completely determines the value of another random variable (think of A as the number of heads in a sequence of coin flips and B as the number of tails in the same sequence of coin flips), the magnitude of the covariance increases. Thus, it stands to reason that if A and B are independent random variables, the covariance between them should be zero, and linearity should hold in this special case. We highly encourage you to work through the math and show this on your own.

Back to the dropout example, the variance of the total number of masked neurons can be broken up into a sum of variances over each neuron, since each neuron is masked independently. The variance of the number of masked neurons is *p(1 – p)*n,* where *p(1 – p)* is the variance for any given neuron and *n* is the number of neurons. Expectation and variance in dropout allow us to understand more deeply what we expect to see when applying such a layer in a deep neural network.

Bayes' Theorem

Returning to our discussion on conditional probability, we noted that the probability of intersection between two events could be written as a product of a conditional distribution and a distribution over a single event. Let's translate this into the language of random variables, now that we have introduced this new terminology. We denote A to be one random variable, and B to denote a second. Let a be a value that A can take on, and b be a value that B can take on. The analogy to the intersection operation for random variables is the *joint probability distribution P(A=a,B=b),* which denotes the event where $A = a$ and $B = b$. We can think of $A = a$ and $B = b$ as individual events, and when we write *P(A = a,B = b)*, we are considering the probability that both events have occurred, i.e., their intersection $P(A = a \cap B = b)$. Note that we generally write the joint probability distribution as *P(A,B),* since this encompasses all possible joint settings of the random variables A and B.

We mentioned earlier that intersection operations could be written as the product of a conditional distribution and a distribution over a single event. Rewriting this in the format for random variables, we have $P(A = a,B = b) = P(A = a|B = b)P(B = b)$. And more generally, considering all possible joint settings of the two random variables, we have $P(A,B) = P(A|B)P(B)$. We also discussed how there always exists a second way of writing this joint distribution as a product: $P(A = a,B = b) = P(B = b|A = a)P(A=a)$, and more generally, $P(A,B) = P(B|A)P(A)$. We noted that sometimes one of these paths makes more sense than the other. For example, in the case where symptoms are represented by A and disease is represented by B, the path in which B takes on a value b, and then A takes on a value a in that universe makes much more sense than the reverse since, biologically, people contract a disease first and only then show symptoms for that disease.

However, this doesn't mean that the reverse isn't useful. It is almost universally the case that people show up at a hospital with mild symptoms, and medical professionals must try to infer the most likely disease from these symptoms to effectively treat the underlying disease. *Bayes' Theorem* gives us a way of calculating the probability of a disease given the observed symptoms. Since the same joint probability distribution can be written in the two ways mentioned in the previous paragraph, we have the following equality:

$$P(B \mid A) = \frac{P(A \mid B)P(B)}{P(A)}$$

If B represents disease, while A represents symptoms, this gives us a method for computing the likelihood of any disease given the observed symptoms. Let's analyze the right side to see if the equality also makes intuitive sense. The likelihood of symptoms given the disease times the likelihood of the disease is just the joint distribution, which makes sense as the numerator here. The denominator is the likelihood of seeing those symptoms, which can also be expressed as a sum of the numerator over all possible diseases. This is an instance of a more general process called *marginalization,* or removing a subset of random variables from a joint distribution by summing over all possible configurations of the subset:

$$P(A) = \Sigma_b P(A, B = b)$$

In more concise terms, we have:

$$P\left(B = b_{query} \mid A\right) = \frac{P\left(B = b_{query}, A\right)}{\Sigma_b P(B = b, A)}$$

Bayes' Theorem is a very valuable application of probability in the real world, especially in the case of disease prediction. Additionally, if we replace the random variable for symptoms with a random variable representing the result of a test for a specific disease, and the random variable over all diseases with a random variable over presence of the specific disease, we can infer the likelihood of actually having a specific disease given a positive test for it using Bayes' Theorem. This is a common problem in most hospitals, and is especially relevant to epidemiology given the outbreak of COVID-19.

Entropy, Cross Entropy, and KL Divergence

Probability distributions, by definition, give us a way of comparing the likelihoods of various possible events. However, even if we know the most likely event (or events) that is to occur, when running the experiment we are bound to see all sorts of events. In this section, we first consider the problem of defining a single metric that encapsulates all of the uncertainty within a probability distribution, which we will define as the distribution's *entropy*.

Let's set up the following scenario. I am a researcher who is running an experiment. The experiment could be something as simple as flipping a coin or rolling a dice. You are recording the results of the experiment. We are both in different rooms, but connected through a phone line. I run the experiment and receive a result, and communicate that result to you via the phone. You record that result in a notebook, where you pick some binary string representation of that result as what you write down. As a scribe, you are necessary in this situation—I may run hundreds of trials and my memory is limited, so I cannot remember the results of all of my trials.

For example, if I roll a dice and neither of us knows anything about the fairness of the dice, you could denote the outcome one as "0," two as "1," three as "10," four as "11," five as "100," and six as "101." Whenever I communicate a result of the experiment to you, you add that result's corresponding string representation to the end of the string consisting of all results so far. If I were to roll a one, followed by two twos, and finally a one, using the encoding scheme defined so far you would have written down "0110."

After all runs of the experiment have ended, I have a meeting with you and try to decipher this string "0110" into a sequence of outcomes for use in my research. However, as the researcher, I am puzzled by this string—does it represent a one, followed by two twos, and finally a one? Or does it represent a one, followed by a two, followed by a three? Or even a one, followed by a four, followed by a one? It seems that there are at least a few possible translations of this string into outcomes using the encoding scheme.

To prevent this situation from ever occurring again, we decide to enforce some limitations on the binary strings you can use to represent outcomes. We use what is called a *prefix code*, which disallows binary string representations of different outcomes from being prefixes of each other. It's not too difficult to see why this would result in a unique translation of string to outcomes. Let's say we have a binary string, some prefix of which we have been able to successfully decode into a series of outcomes. To decode the rest of the string, or the suffix, we must first find the next outcome in the series. When we find a prefix of this suffix that translates to an outcome, we already know that, by definition, there is no smaller prefix that translates to a valid outcome. We now have a larger prefix of the binary string that has been successfully translated to a series of outcomes. We then recursively use this logic until we have reached the end of the string.

Now that we have some guidelines on string representations for outcomes, we redo the original experiment with one as "0," two as "10," three as "110," four as "1110," five as "11110," and six as "111110." However, as noted earlier, I may carry out hundreds of trials, and as the scribe you probably want to limit the amount of writing you have to do. With no information about the dice, we can't do too much better than this. Assuming each outcome shows up with probability $\frac{1}{6}$, the expected number of letters you'd need to write down per trial is 3.5. We could get down to 3 if we set one as "000," two as "001," three as "010," four as "011," five as "100," and six as "101," for example.

But what if we knew information about the dice? For example, what if it were a weighted dice that showed up six almost all of the time? In that case, you probably want to assign a shorter binary string to six, for example "0" (instead of assigning "0" to one) so you can limit the expected amount of writing you have to do. It makes intuitive sense that, as the result of any single trial becomes more and more certain, the expected number of characters you'd need to write becomes lower by assigning the shortest binary strings to the most likely outcomes.

This raises the question: given a probability distribution over outcomes, what is the optimal encoding scheme, where optimal is defined as the fewest expected number of characters you'd need to write per trial? Although this whole situation may feel a bit contrived, it provides us with a slightly different lens through which we can understand the uncertainty within a probability distribution. As we noted, as the result of an experiment becomes more and more certain, the optimal encoding scheme would allow the scribe to write fewer and fewer characters in expectation per trial. For example, in the extreme case where we already knew beforehand that a six would always show up, the scribe wouldn't need to write anything down.

It turns out that, although we won't show it here, the best you can do is assign a binary string of length $\log_2 \frac{1}{p(x_i)}$ to each possible outcome x_i, where $p(x_i)$ is its probability. The expected string length of any given trial would then be:

$$\mathbb{E}_{p(x)}\left[\log_2 \frac{1}{p(x)} \right] = \Sigma_{x_i} p(x_i) \log_2 \frac{1}{p(x_i)}$$
$$= -\Sigma_{x_i} p(x_i) \log_2 p(x_i)$$

This expression is defined as the *entropy* of a probability distribution. In the case where we are completely certain of the final outcome (e.g., the dice always lands up six), we can evaluate the expression for entropy and see that we get a result of 0.

In the case where we are completely certain of the final outcome (e.g., the dice always lands up six), we can evaluate the expression for entropy and see that we get a result of 0. Additionally, the probability distribution that has the highest entropy is the one that places equal probability over all possible outcomes. This is because, for any given trial, we are no more certain that a particular outcome will appear as opposed to any other outcome. As a result, we cannot use the strategy of assigning a shorter string to any single outcome.

Now that we have defined entropy, we can discuss cross entropy, which provides us a way of measuring the distinctness of two distributions.

Equation 2-1. Cross entropy

$$CE(p \,||\, q) = \mathbb{E}_{p(x)}\left[\log_2 \frac{1}{q(x)} \right] = \Sigma_x p(x) \log_2 \frac{1}{q(x)} = -\Sigma_x p(x) \log_2 q(x)$$

Note that cross entropy has a $\log \frac{1}{q(x)}$ term, which can be interpreted as the optimal binary string length assigned to each outcome, assuming outcomes appear according to probability distribution *q(x)*. However, note that this is an expectation with respect to *p(x)*, so how do we interpret this entire expression? Well, we can understand the cross entropy to mean the expected string length for any trial given we have optimized for the encoding scheme for distribution *q(x)* while, in reality, all of the outcomes are appearing according to the distribution *p(x)*. This can definitely happen in an experiment where we have only limited a priori information about the experiment, so we assume some distribution *q(x)* to optimize our encoding scheme, but as we carry out trials, we learn more information that gets us closer to the true distribution *p(x)*.

The KL divergence takes this logic a bit further. If we take the cross entropy, which tells us the expected number of bits per trial given we have optimized our encoding for the incorrect distribution *q(x)*, and subtract from that the entropy, which tells

us the expected number of bits per trial given we have optimized for the correct distribution $p(x)$, we get the expected number of extra bits required to represent a trial when using $q(x)$ compared to $p(x)$. Here is the expression for the KL divergence:

$$KL(p||q) = \mathbb{E}_{p(x)}\left[\log_2 \frac{1}{q(x)} - \log_2 \frac{1}{p(x)}\right] = \mathbb{E}_{p(x)}\left[\log_2 \frac{p(x)}{q(x)}\right]$$

At the unique global minimum $q(x) = p(x)$, the KL divergence is exactly zero. Why this is the unique minimum is a bit beyond the scope of this text, so we leave that as an exercise for you.

In practice, when trying to match the true distribution $p(x)$ with a learned distribution $q(x)$, KL divergence is often minimized as an objective function. Most models will actually minimize the cross entropy in place of the KL divergence, which is effectively the same optimization problem due to the KL being a difference between the cross entropy and the entropy of $p(x)$, where the entropy of $p(x)$ is a constant and has no dependence on the weights that parameterize $q(x)$. Thus, the gradient with respect to the weights that parameterize $q(x)$ when using either objective is the same.

One common example where cross-entropy/KL divergence is optimized is in the standard training of a neural network classifier. The neural network's objective is to learn a distribution over target classes such that, for any given example x_i, $p_\theta(y|x = x_i)$ matches the true distribution $p(y|x = x_i)$, which has all of its probability mass placed over the true label y_i and zero probability over all other classes. Minimizing the sum of cross entropies between the learned distribution and the true distribution over all examples is actually the exact same as minimizing the negative log likelihood of the data. Both are valid interpretations of how neural networks are trained, and lead to the same objective function. We encourage you to try writing out both expressions independently to see this.

Continuous Probability Distributions

So far, we have looked at probability distributions through the lens of discrete outcomes and events. However, as it turns out, probability distributions aren't just for sets of discrete outcomes like the CIFAR-10 target classes or the MNIST digits. We can define probability distributions over sample spaces of infinite size, such as all the real numbers. In this section, we will extend principles covered in the previous sections to the continuous realm.

In the continuous realm, probability distributions are often referred to as *probability density functions*, or PDFs. PDFs are nonnegative functions over a sample space, such as all the reals, that integrate to one. Recall from calculus that the integration of a function is the area of the region underneath the function, bounded by the x-axis. PDFs follow the basic tenets introduced in the first section, but instead of adding

the probability of outcomes to get the probability of an event, we use integration. For example, say X is a continuous random variable that is defined over all the real numbers. If we'd like to know the probability of the event $P(X \leq 2)$, all we'd need to do is integrate the PDF of X from negative infinity to 2.

But how about the probability of any individual outcome, say $P(X = 2)$? Since we use integration to find probabilities in the continuous space, the probability of any individual outcome is actually zero due to the width of the region being infinitesimal. We instead use the term *likelihood* to distinguish between the probability of events and the value that the PDF evaluates to when we input a setting of X. Likelihoods are still valuable, as they tell us what individual outcomes we are most likely to see when performing an experiment over a continuous space. Going forward, when considering continuous probability distributions, we will only refer to events as having probability, rather than individual outcomes.

One famous example of a continuous probability distribution is the *uniform distribution* over some interval on the real line. Under the uniform distribution, the likelihood of each outcome is the same, meaning that no outcome is any more likely to appear than another. Thus, the uniform distribution looks like a rectangle, where the base of the rectangle is the interval constituting its domain, and the height, or the likelihood for each outcome, is the value that makes the area of the rectangle equal to one. Figure 2-4 shows the uniform distribution over the interval [0,0.5].

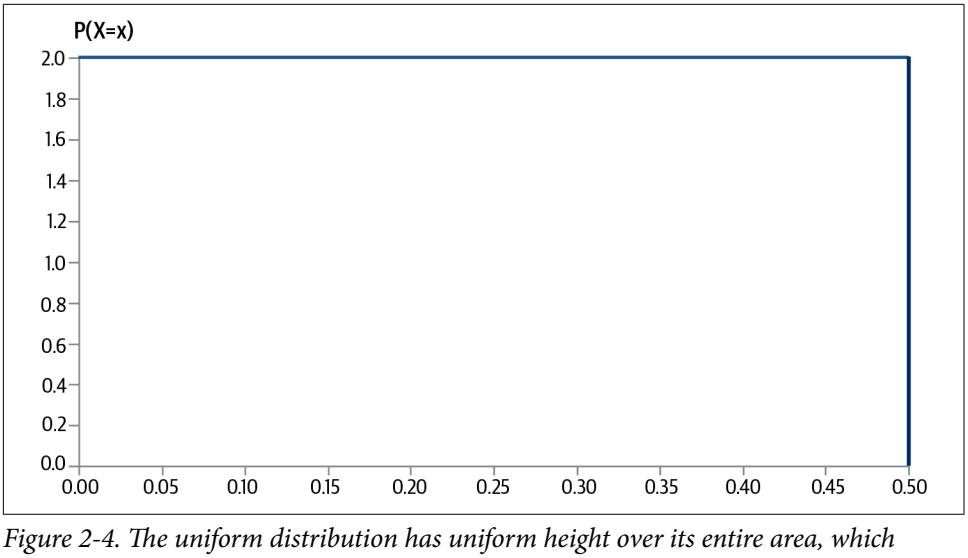


Figure 2-4. The uniform distribution has uniform height over its entire area, which shows that each value in the domain of the distribution has equal likelihood.

This example was chosen specifically to show a concrete difference between likelihoods and probabilities in the continuous realm. The height of the rectangle being

2 was no error—there is no constraint on the magnitude of the likelihood in continuous distributions, unlike probabilities, which must be less than or equal to 1.

Another famous example of a continuous probability distribution is the *Gaussian distribution*, which is one of the more common ways in which data presents itself in the real world. The Gaussian distribution is defined by two parameters: its mean μ and its standard deviation σ. The PDF of a Gaussian distribution is:

$$f(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

Why this function integrates to 1 over the real domain is beyond the scope of this chapter, but one important characteristic of a Gaussian distribution is that its mean is also its unique mode. In other words, the outcome with the highest likelihood is also, uniquely, the mean outcome. This is not the case for all distributions. For example, Figure 2-4 does not have this property. The graph of a standard Gaussian, which has mean zero and unit variance, is shown in Figure 2-5 (the PDF asymptotically reaches zero in the limit in both directions).

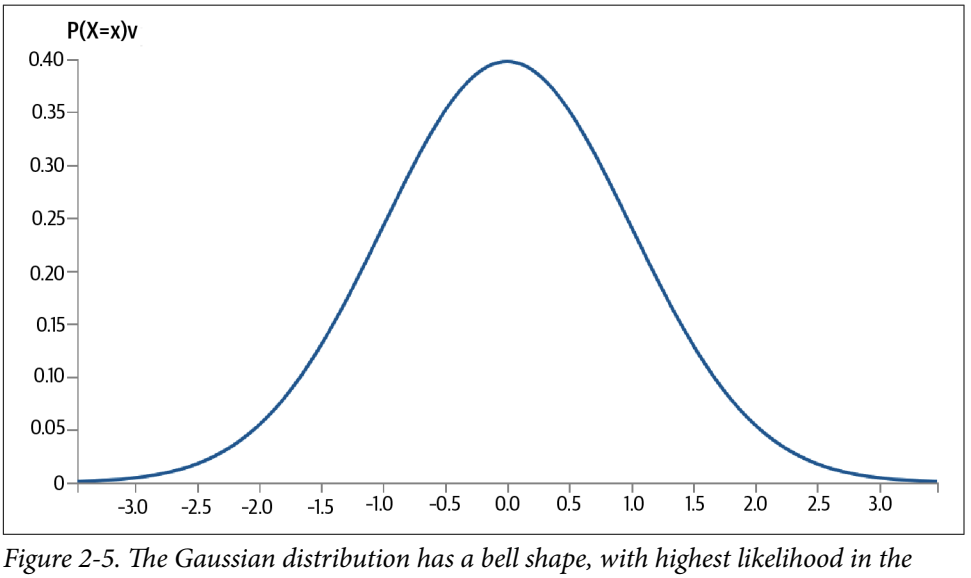


Figure 2-5. The Gaussian distribution has a bell shape, with highest likelihood in the center and dropping exponentially as the value in question gets farther and farther from the center.

Why is the Gaussian distribution so prevalent in real-world data? One reason for this is a theorem called the *Central Limit Theorem* (CLT). This theorem states that sums of independent random variables converge to a Gaussian distribution as the number of variables in the sum goes to infinity, even if each variable is not distributed as

a Gaussian. One example is the number of masked neurons after a dropout layer is applied. As the number of neurons from the previous layer goes to infinity, the number of masked neurons (which is a sum of independent Bernoulli random variables, as discussed in "Random Variables" on page 22), when standardized correctly, is approximately distributed as a standard Gaussian distribution. We won't cover CLT in much depth here, but it has more recently been extended to weakly dependent variables under certain special conditions.

Many real-world datasets can be seen as approximately sums of many random variables. For example, the distribution of a disease prevalence within a given population, similarly to the number of masked neurons after applying dropout, is the sum of many Bernoulli random variables (where each person is a Bernoulli random variable that has a value of 1 if they have the disease and a value of 0 if they do not)—although likely dependent.

Continuous random variables are still functions, just as we defined discrete random variables. The only difference is that the range of this function is a continuous space. To compute the expectation and variance of a continuous random variable, all we need to do is replace our summations with integrations, as follows:

$$\mathbb{E}[X] = \int_x x * f(X = x)dx$$

$$Var(X) = \int_x (x - \mathbb{E}[X])^2 * f(X = x)dx$$

As an example, let's evaluate the expectation for our uniform random variable defined earlier. But first, confirm that it makes intuitive sense that the expectation should be 0.25, since the endpoints of the interval are 0 and 0.5 and all values in between are of equal likelihood. Now, let's evaluate the integral and see if the computation matches our intuition:

$$\int_0^{0.5} x * f(x)dx = \int_0^{0.5} 2xdx$$

$$= x^2\Big|_0^{0.5}$$

$$= 0.25$$

Where the superscript and the subscript of the | symbol represent the values at which we will evaluate the preceding function, which we will then difference to get the value of the integral. We see that the expectation comes out to the same value as our intuition, which is a great sanity check.

Bayes' Theorem also holds for continuous variables. The only major difference is when marginalizing out a subset of variables, you will need to integrate over the entire domain of the marginalized subset rather than taking a discrete sum over

all possible configurations of the marginalized subset. Again, this is an example of extending the tenets of probability to the continuous space by replacing summations with integrations. Here is Bayes' Theorem for continuous probability distributions, following the notation from "Bayes' Theorem" on page 27:

$$P\left(B = b_{query} \middle| A\right) = \frac{P\left(A \middle| B = b_{query}\right)P\left(B = b_{query}\right)}{P(A)} = \frac{P\left(A \middle| B = b_{query}\right)P\left(B = b_{query}\right)}{\int_b P(A, B = b)db}$$

And finally, we have our discussion on entropy, cross entropy, and KL divergence. All three of these extend nicely to the continuous space as well. We replace our summations with integrations and note that the properties introduced in the previous section still hold. For example, over a given domain, the distribution with the highest entropy is the uniform distribution, and the KL divergence between two distributions is zero if and only if the two distributions are the exact same. Here are the definitions in their continuous form, following Equation 2-1:

$$H(f(x)) = -\int_x f(x)\log_2 f(x)dx$$

$$KL(f(x)||g(x)) = \int_x f(x)\log_2 \frac{f(x)}{g(x)}dx$$

$$CE(f(x)||g(x)) = -\int_x f(x)\log_2 g(x)dx$$

Our extension of these concepts to the continuous space will come in handy in Chapter 10, where we model many distributions as Gaussians. Additionally, we use the KL divergence/cross-entropy terms as a regularization procedure on the complexity of one of our learned distributions. Since KL divergence is only zero when the query distribution matches the target distribution, setting the target distribution to a Gaussian forces the learned distribution to approximate a Gaussian.

Summary

In this chapter we covered the fundamentals of probability, first building the intuition behind the basics of probability distributions and then moving to relevant applications of probability, such as conditional probability, random variables, expectation, and variance. We saw the applications of probability in deep learning, such as how a neural net parametrizes a probability distribution during classification tasks, and how we can quantify the mathematical properties of dropout, a regularization technique in neural nets. Finally, we discussed measurements of uncertainty in probability distributions such as entropy, and generalized these concepts to the continuous realm.

Probability is a field that affects the choices in our everyday lives, and it's key to understand the meaning behind the numbers. Additionally, we hope that this introduction puts the rest of the book in perspective and allows you to more rigorously understand future concepts. In the next chapter, we will discuss the structure of neural networks, and the motivations behind their design.

CHAPTER 3

The Neural Network

Building Intelligent Machines

The brain is the most incredible organ in the human body. It dictates the way we perceive every sight, sound, smell, taste, and touch. It enables us to store memories, experience emotions, and even dream. Without it, we would be primitive organisms, incapable of anything other than the simplest of reflexes. The brain is, inherently, what makes us intelligent.

The infant brain weighs only a single pound, but somehow it solves problems that even our biggest, most powerful supercomputers find impossible. Within a matter of months after birth, infants can recognize the faces of their parents, discern discrete objects from their backgrounds, and even tell voices apart. Within a year, they've already developed an intuition for natural physics, can track objects even when they become partially or completely blocked, and can associate sounds with specific meanings. And by early childhood, they have a sophisticated understanding of grammar and thousands of words in their vocabularies.[1]

For decades, we've dreamed of building intelligent machines with brains like ours—robotic assistants to clean our homes, cars that drive themselves, microscopes that automatically detect diseases. But building these artificially intelligent machines requires us to solve some of the most complex computational problems we have ever grappled with; problems that our brains can already solve in a manner of microseconds. To tackle these problems, we'll have to develop a radically different way of programming a computer using techniques largely developed over the past decade. This is an extremely active field of artificial computer intelligence often referred to as *deep learning*.

1 Kuhn, Deanna, et al. *Handbook of Child Psychology. Vol. 2, Cognition, Perception, and Language*. Wiley, 1998.

The Limits of Traditional Computer Programs

Why exactly are certain problems so difficult for computers to solve? Well, it turns out that traditional computer programs are designed to be very good at two things: (1) performing arithmetic really fast and (2) explicitly following a list of instructions. So if you want to do some heavy financial number crunching, you're in luck. Traditional computer programs can do the trick. But let's say we want to do something slightly more interesting, like write a program to automatically read someone's handwriting. Figure 3-1 will serve as a starting point.

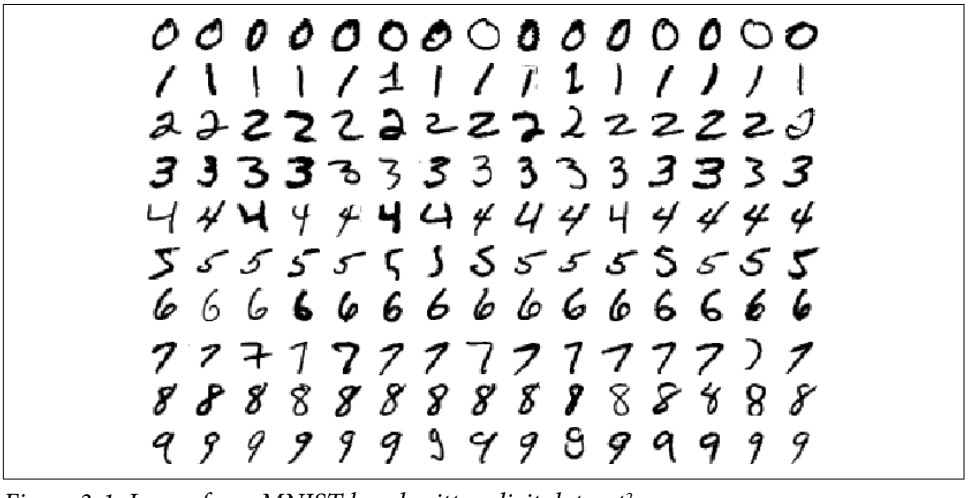


Figure 3-1. Image from MNIST handwritten digit dataset[2]

Although every digit in Figure 3-1 is written in a slightly different way, we can easily recognize every digit in the first row as a zero, every digit in the second row as a one, etc. Let's try to write a computer program to crack this task. What rules could we use to tell one digit from another?

Well, we can start simple! For example, we might state that we have a zero if our image has only a single, closed loop. All the examples in Figure 3-1 seem to fit this bill, but this isn't really a sufficient condition. What if someone doesn't perfectly close the loop on their zero? And, as in Figure 3-2, how do you distinguish a messy zero from a six?

2 Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE*, 86(11):2278-2324, November 1998.

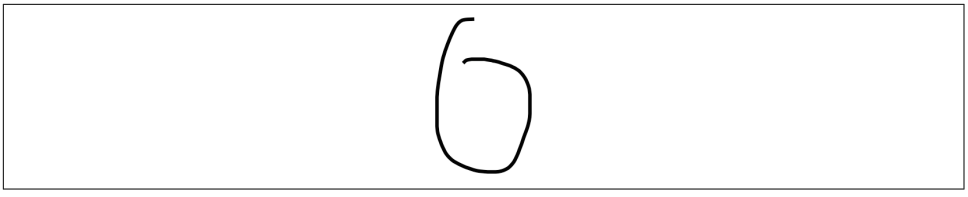


Figure 3-2. A zero that's algorithmically difficult to distinguish from a six

You could potentially establish some sort of cutoff for the distance between the starting point of the loop and the ending point, but it's not exactly clear where we should be drawing the line. But this dilemma is only the beginning of our worries. How do we distinguish between threes and fives? Or between fours and nines? We can add more and more rules, or *features*, through careful observation and months of trial and error, but it's quite clear that this isn't going to be an easy process.

Many other classes of problems fall into this same category: object recognition, speech comprehension, automated translation, etc. We don't know what program to write because we don't know how it's done by our brains. And even if we did know how to do it, the program might be horrendously complicated.

The Mechanics of Machine Learning

To tackle these classes of problems, we'll have to use a different kind of approach. A lot of the things we learn in school growing up have much in common with traditional computer programs. We learn how to multiply numbers, solve equations, and take derivatives by internalizing a set of instructions. But the things we learn at an extremely early age, the things we find most natural, are learned by example, not by formula.

For instance, when we were two years old, our parents didn't teach us how to recognize a dog by measuring the shape of its nose or the contours of its body. We learned to recognize a dog by being shown multiple examples and being corrected when we made the wrong guess. When we were born, our brains provided us with a model that described how we would be able to see the world. As we grew up, that model would take in our sensory inputs and make a guess about what we were experiencing. If that guess was confirmed by our parents, our model would be reinforced. If our parents said we were wrong, we'd modify our model to incorporate this new information. Over our lifetime, our model becomes more and more accurate as we assimilate more and more examples. Obviously all of this happens subconsciously, but we can use this to our advantage.

Deep learning is a subset of a more general field of AI called *machine learning*, which is predicated on this idea of learning from example. In machine learning, instead of teaching a computer a massive list of rules to solve the problem, we give it a *model*

with which it can evaluate examples, and a small set of instructions to modify the model when it makes a mistake. We expect that, over time, a well-suited model would be able to solve the problem extremely accurately.

Let's be a little bit more rigorous about what this means so we can formulate this idea mathematically. Let's define our model to be a function $h(\mathbf{x}, \theta)$. The input $\mathbf{x}$ is an example expressed in vector form. For example, if $\mathbf{x}$ were a grayscale image, the vector's components would be pixel intensities at each position, as shown in Figure 3-3.

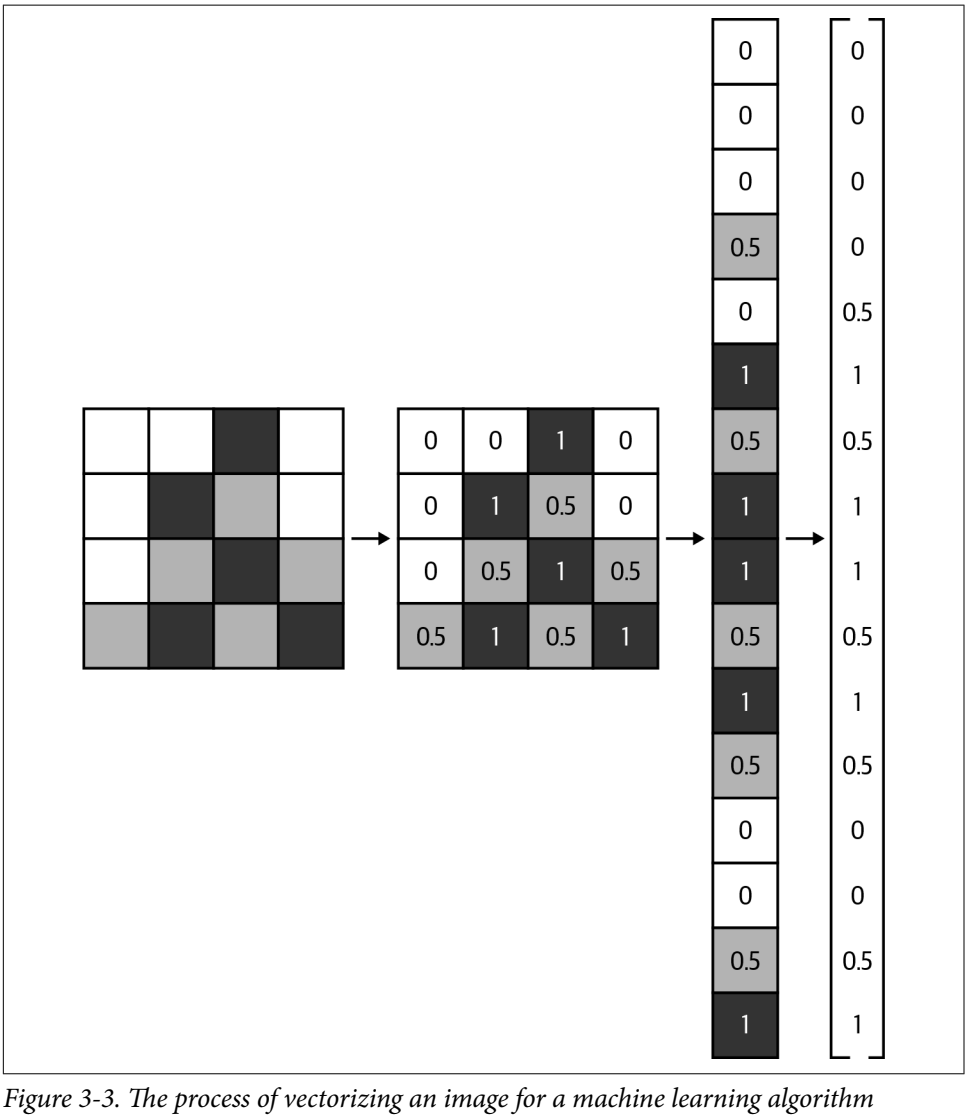


Figure 3-3. The process of vectorizing an image for a machine learning algorithm

The input θ is a vector of the parameters that our model uses. Our machine learning program tries to perfect the values of these parameters as it is exposed to more and more examples. We'll see this in action and in more detail in Chapter 4.

To develop a more intuitive understanding for machine learning models, let's walk through a quick example. Let's say we wanted to determine how to predict exam performance based on the number of hours of sleep we get and the number of hours we study the previous day. We collect a lot of data, and for each data point $\mathbf{x} = \begin{bmatrix} x_1 & x_2 \end{bmatrix}^T$, we record the number of hours of sleep we got (x_1), the number of hours we spent studying (x_2), and whether we performed above or below the class average. Our goal, then, might be to learn a model $h(\mathbf{x}, \theta)$ with parameter vector $\theta = \begin{bmatrix} \theta_0 & \theta_1 & \theta_2 \end{bmatrix}^T$ such that:

$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } \mathbf{x}^T \cdot \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \theta_0 < 0 \\ 1 & \text{if } \mathbf{x}^T \cdot \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} + \theta_0 \geq 0 \end{cases}$$

So we guess that the blueprint for our model $h(\mathbf{x}, \theta)$ is as described (geometrically, this particular blueprint describes a linear classifier that divides the coordinate plane into two halves). Then, we want to learn a parameter vector θ such that our model makes the right predictions (–1 if we perform below average, and 1 otherwise) given an input example $\mathbf{x}$. This model is called a linear *perceptron*, and it's a model that's been used since the 1950s.[3] Let's assume our data is as shown in Figure 3-4.

3 Rosenblatt, Frank. "The perceptron: A Probabilistic Model for Information Storage and Organization in the Brain." *Psychological Review* 65.6 (1958): 386.

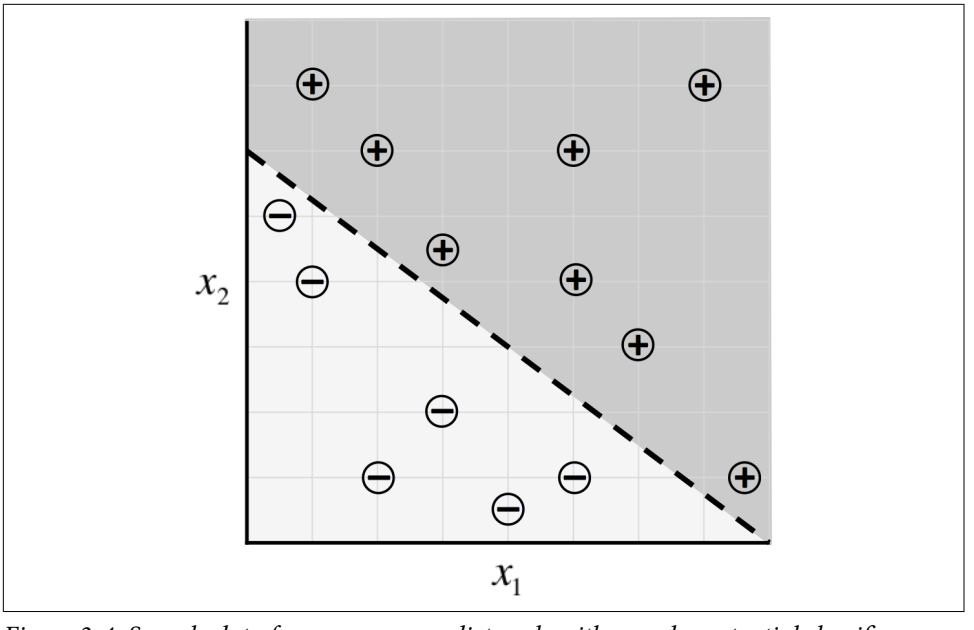


Figure 3-4. Sample data for our exam predictor algorithm and a potential classifier

Then it turns out that by selecting $\theta = [-24\ 3\ 4]^T$, our machine learning model makes the correct prediction on every data point:

$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } 3x_1 + 4x_2 - 24 < 0 \\ 1 & \text{if } 3x_1 + 4x_2 - 24 \geq 0 \end{cases}$$

An optimal parameter vector θ positions the classifier so that we make as many correct predictions as possible. In most cases, there are many (or even infinitely many) possible choices for θ that are optimal. Fortunately for us, most of the time these alternatives are so close to one another that the difference is negligible. If this is not the case, we may want to collect more data to narrow our choice of θ.

While the setup seems reasonable, there are still some pretty significant questions that remain. First off, how do we even come up with an optimal value for the parameter vector θ in the first place? Solving this problem requires a technique commonly known as *optimization*. An optimizer aims to maximize the performance of a machine learning model by iteratively tweaking its parameters until the error is minimized. We'll begin to tackle this question of learning parameter vectors in

more detail in Chapter 4, when we describe the process of *gradient descent*.[4] In later chapters, we'll try to find ways to make this process even more efficient.

Second, it's quite clear that this particular model (the linear perceptron model) is quite limited in the relationships it can learn. For example, the distributions of data shown in Figure 3-5 cannot be described well by a linear perceptron.

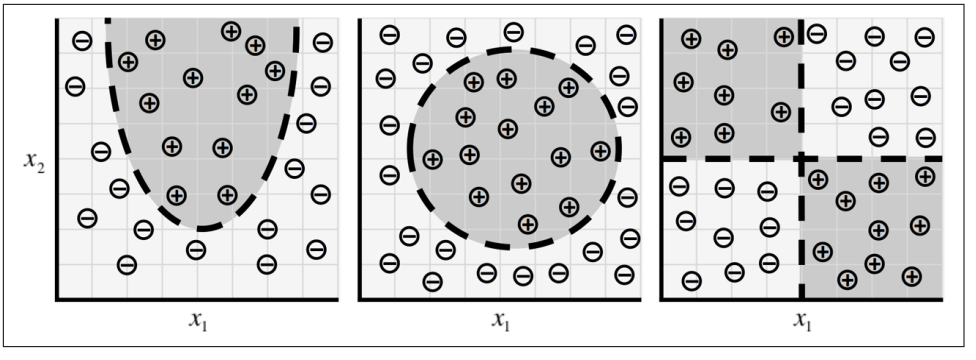


Figure 3-5. As our data takes on more complex forms, we need more complex models to describe them

But these situations are only the tip of the iceberg. As we move on to much more complex problems, such as object recognition and text analysis, our data becomes extremely high dimensional, and the relationships we want to capture become highly nonlinear. To accommodate this complexity, recent research in machine learning has attempted to build models that resemble the structures utilized by our brains. It's essentially this body of research, commonly referred to as *deep learning*, that has had spectacular success in tackling problems in computer vision and natural language processing. These algorithms not only far surpass other kinds of machine learning algorithms, but also rival (or even exceed) the accuracies achieved by humans.

The Neuron

The foundational unit of the human brain is the neuron. A tiny piece of the brain, about the size of grain of rice, contains over 10,000 neurons, each of which forms an average of 6,000 connections with other neurons.[5] It's this massive biological network that enables us to experience the world around us. Our goal in this section is to use this natural structure to build machine learning models that solve problems in an analogous way.

4 Bubeck, Sébastien. "Convex Optimization: Algorithms and Complexity." *Foundations and Trends® in Machine Learning*. 8.3-4 (2015): 231-357.

5 Restak, Richard M. and David Grubin. *The Secret Life of the Brain*. Joseph Henry Press, 2001.

At its core, the neuron is optimized to receive information from other neurons, process this information in a unique way, and send its result to other cells. This process is summarized in Figure 3-6. The neuron receives its inputs along antennae-like structures called *dendrites*. Each of these incoming connections is dynamically strengthened or weakened based on how often it is used (this is how we learn new concepts), and it's the strength of each connection that determines the contribution of the input to the neuron's output. After being weighted by the strength of their respective connections, the inputs are summed together in the *cell body*. This sum is then transformed into a new signal that's propagated along the cell's *axon* and sent off to other neurons.

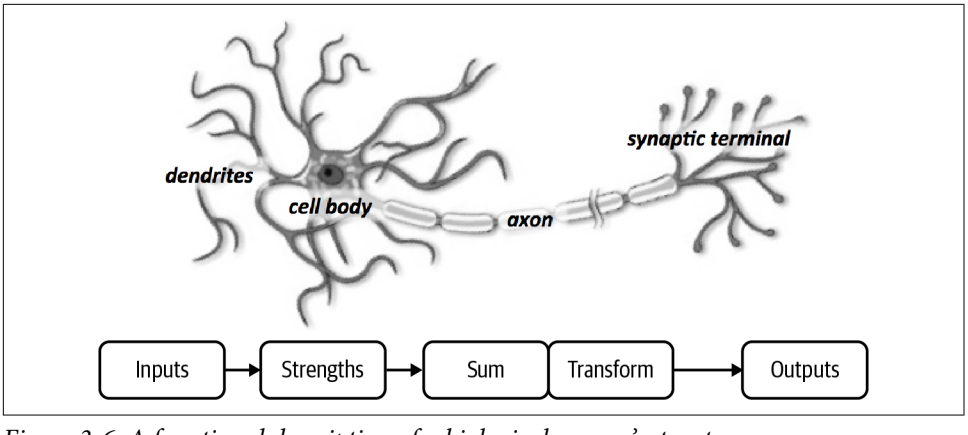


Figure 3-6. A functional description of a biological neuron's structure

We can translate this functional understanding of the neurons in our brain into an artificial model that we can represent on our computer. Such a model is described in Figure 3-7, leveraging the approach first pioneered in 1943 by Warren S. McCulloch and Walter H. Pitts.[6] Just as in biological neurons, our artificial neuron takes in some number of inputs, $x_1, x_2, ..., x_n$, each of which is multiplied by a specific weight, $w_1, w_2, ..., w_n$. These weighted inputs are, as before, summed to produce the *logit* of the neuron, $z = \sum_{i=0}^{n} w_i x_i$. In many cases, the logit also includes a *bias*, which is a constant (not shown in the figure). The logit is then passed through a function f to produce the output $y = f(z)$. This output can be transmitted to other neurons.

6 McCulloch, Warren S., and Walter Pitts. "A Logical Calculus of the Ideas Immanent in Nervous Activity." *The Bulletin of Mathematical Biophysics*. 5.4 (1943): 115-133.

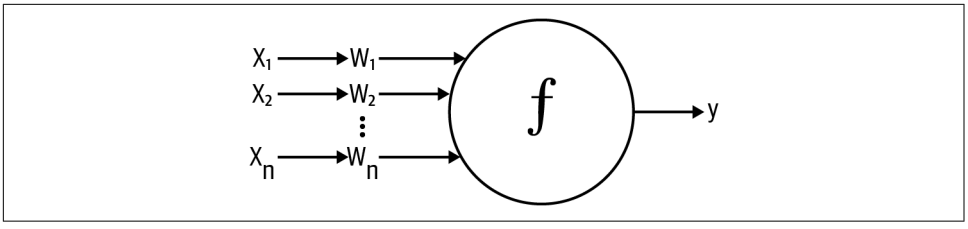


Figure 3-7. Schematic for a neuron in an artificial neural net

We'll conclude our mathematical discussion of the artificial neuron by re-expressing its functionality in vector form. Let's reformulate the inputs as a vector $\mathbf{x} = [x_1\ x_2\ \ldots\ x_n]$ and the weights of the neuron as $\mathbf{w} = [w_1\ w_2\ \ldots\ w_n]$. Then we can re-express the output of the neuron as $y = f(\mathbf{x} \cdot \mathbf{w} + b)$, where b is the bias term. We can compute the output by performing the dot product of the input and weight vectors, adding in the bias term to produce the logit, and then applying the transformation function. While this seems like a trivial reformulation, thinking about neurons as a series of vector manipulations will be crucial to how we implement them in software later in this book.

Expressing Linear Perceptrons as Neurons

In "The Mechanics of Machine Learning" on page 41, we talked about using machine learning models to capture the relationship between success on exams and time spent studying and sleeping. To tackle this problem, we constructed a linear perceptron classifier that divided the Cartesian coordinate plane into two halves:

$$h(\mathbf{x}, \theta) = \begin{cases} -1 & \text{if } 3x_1 + 4x_2 - 24 < 0 \\ 1 & \text{if } 3x_1 + 4x_2 - 24 \geq 0 \end{cases}$$

As shown in Figure 3-4, this is an optimal choice for θ because it correctly classifies every sample in our dataset. Here, we show that our model h is easily using a neuron. Consider the neuron depicted in Figure 3-8. The neuron has two inputs, a bias, and uses the function:

$$f(z) = \begin{cases} -1 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

It's easy to show that our linear perceptron and the neuronal model are perfectly equivalent. And in general, it's quite simple to show that singular neurons are strictly more expressive than linear perceptrons. Every linear perceptron can be expressed as a single neuron, but single neurons can also express models that cannot be expressed by any linear perceptron.

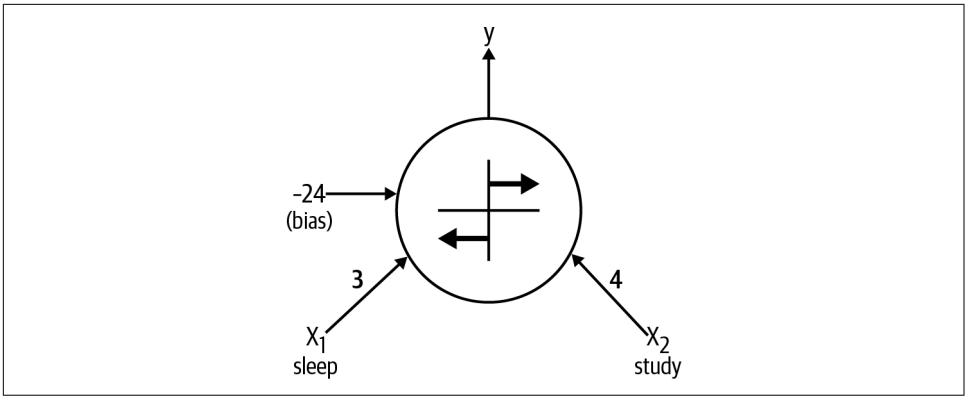


Figure 3-8. Expressing our exam performance perceptron as a neuron

Feed-Forward Neural Networks

Although single neurons are more powerful than linear perceptrons, they're not nearly expressive enough to solve complicated learning problems. There's a reason our brain is made of more than one neuron. For example, it is impossible for a single neuron to differentiate handwritten digits. So to tackle much more complicated tasks, we'll have to take our machine learning model even further.

The neurons in the human brain are organized in layers. In fact, the human cerebral cortex (the structure responsible for most of human intelligence) is made up of six layers.[7] Information flows from one layer to another until sensory input is converted into conceptual understanding. For example, the bottommost layer of the visual cortex receives raw visual data from the eyes. This information is processed by each layer and passed on to the next until, in the sixth layer, we conclude whether we are looking at a cat, or a soda can, or an airplane.

Borrowing from these concepts, we can construct an *artificial neural network*. A neural network comes about when we start hooking up neurons to each other, the input data, and to the output nodes, which correspond to the network's answer to a learning problem. Figure 3-9 demonstrates a simple example of an artificial neural network, similar to the architecture described in McCulloch and Pitt's work in 1943.

7 Mountcastle, Vernon B. "Modality and Topographic Properties of Single Neurons of Cat's Somatic Sensory Cortex." *Journal of Neurophysiology* 20.4 (1957): 408-434.

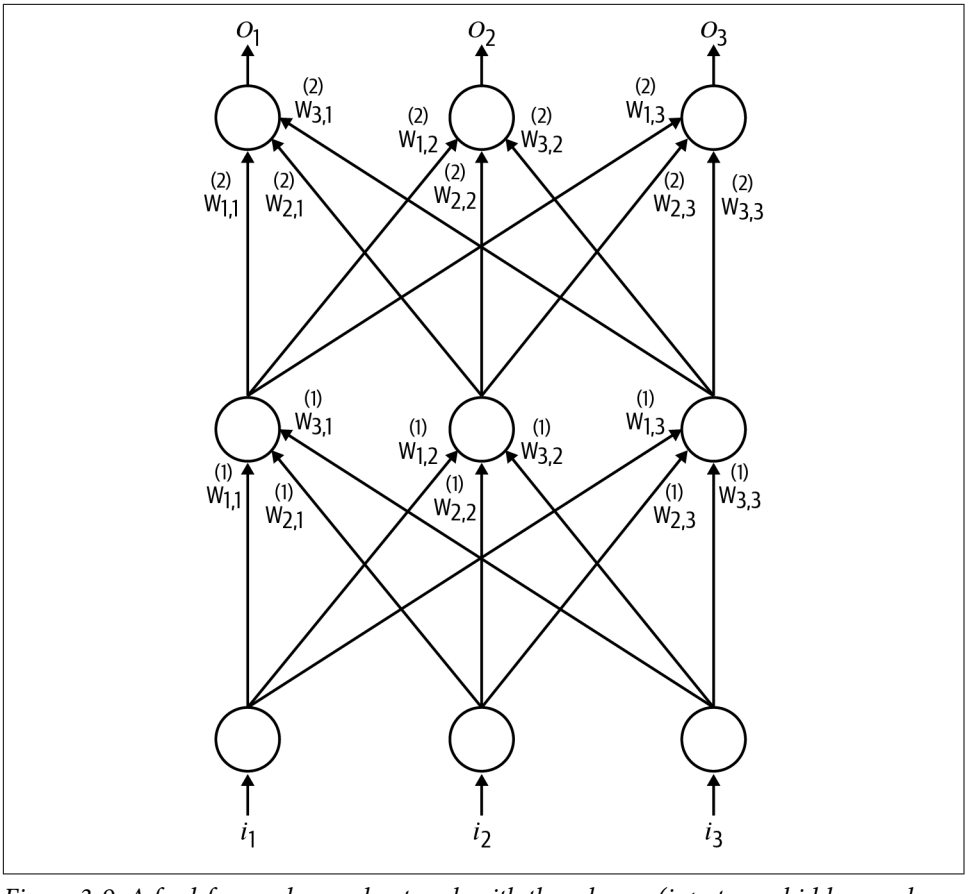


Figure 3-9. A feed-forward neural network with three layers (input, one hidden, and output) and three neurons per layer

The bottom layer of the network pulls in the input data. The top layer of neurons (output nodes) computes our final answer. The middle layer(s) of neurons are called the *hidden layers*, and we let $w_{i,j}^{(k)}$ be the weight of the connection between the ith neuron in the kth layer with the jth neuron in the $k+1^{st}$ layer. These weights constitute our parameter vector, θ, and just as before, our ability to solve problems with neural networks depends on finding the optimal values to plug into θ.

We note that in this example, connections traverse only from a lower layer to a higher layer. There are no connections between neurons in the same layer, and there are no connections that transmit data from a higher layer to a lower layer. These neural networks are called *feed-forward* networks, and we start by discussing these networks because they are the simplest to analyze. We present this analysis

(specifically, the process of selecting the optimal values for the weights) in Chapter 4. More complicated connectivities will be addressed in later chapters.

We'll discuss the major types of layers that are utilized in feed-forward neural networks, but before we proceed, here's a couple of important notes to keep in mind:

1. As we mentioned, the layers of neurons that lie sandwiched between the first layer of neurons (input layer) and the last layer of neurons (output layer) are called the hidden layers. This is where most of the magic is happening when the neural net tries to solve problems. Whereas (as in the handwritten digit example) we would previously have to spend a lot of time identifying useful features, the hidden layers automate this process for us. Oftentimes, taking a look at the activities of hidden layers can tell you a lot about the features the network has automatically learned to extract from the data.
2. Although in Figure 3-9 every layer has the same number of neurons, this is neither necessary nor recommended. More often than not, hidden layers have fewer neurons than the input layer to force the network to learn compressed representations of the original input. For example, while our eyes obtain raw pixel values from our surroundings, our brain thinks in terms of edges and contours. This is because the hidden layers of biological neurons in our brain, force us to come up with better representations for everything we perceive.
3. It is not required that every neuron has its output connected to the inputs of all neurons in the next layer. In fact, selecting which neurons to connect to which other neurons in the next layer is an art that comes from experience. We'll discuss this issue in more depth as we work through various examples of neural networks.
4. The inputs and outputs are *vectorized* representations. For example, you might imagine a neural network where the inputs are the individual pixel RGB values in an image represented as a vector (refer to Figure 3-3). The last layer might have two neurons that correspond to the answer to our problem: $[1, 0]$ if the image contains a dog, $[0, 1]$ if the image contains a cat, $[1, 1]$ if it contains both, and $[0, 0]$ if it contains neither.

We'll also observe that, similarly to our reformulation for the neuron, we can also mathematically express a neural network as a series of vector and matrix operations. Let's consider the input to the ith layer of the network to be a vector $\mathbf{x} = [x_1\ x_2\ \ldots\ x_n]$. We'd like to find the vector $\mathbf{y} = [y_1\ y_2\ \ldots\ y_m]$ produced by propagating the input through the neurons. We can express this as a simple matrix multiply if we construct a weight matrix $\mathbf{W}$ of size $n \times m$ and a bias vector of size m. In this matrix, each column corresponds to a neuron, where the jth element of the column corresponds to the weight of the connection pulling in the jth element of the input. In other words, $\mathbf{y} = f(\mathbf{W}^T\mathbf{x} + \mathbf{b})$, where the transformation function is applied to the vector

element-wise. This reformulation will become all the more critical as we begin to implement these networks in software.

Linear Neurons and Their Limitations

Most neuron types are defined by the function f they apply to their logit z. Let's first consider layers of neurons that use a linear function in the form of $f(z) = az + b$. For example, a neuron that attempts to estimate a cost of a meal in a fast-food restaurant would use a linear neuron where $a = 1$ and $b = 0$. Using $f(z) = z$ and weights equal to the price of each item, the linear neuron in Figure 3-10 would take in some ordered triple of servings of burgers, fries, and sodas, and output the price of the combination.

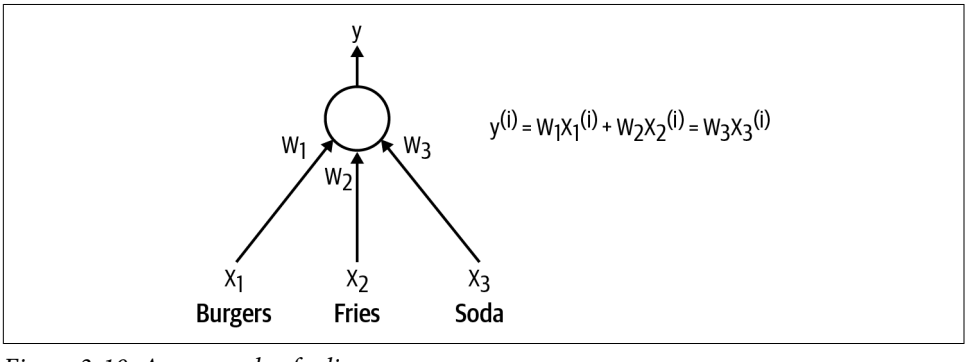


Figure 3-10. An example of a linear neuron

Linear neurons are easy to compute with, but they run into serious limitations. In fact, it can be shown that any feed-forward neural network consisting of only linear neurons can be expressed as a network with no hidden layers. This is problematic because, as we discussed, hidden layers are what enable us to learn important features from the input data. In other words, to learn complex relationships, we need to use neurons that employ some sort of nonlinearity.

Sigmoid, Tanh, and ReLU Neurons

Three major types of neurons are used in practice that introduce nonlinearities in their computations. The first of these is the *sigmoid neuron*, which uses the function:

$$f(z) = \frac{1}{1 + e^{-z}}$$

Intuitively, this means that when the logit is very small, the output of a logistic neuron is close to 0. When the logit is very large, the output of the logistic neuron is

close to 1. In-between these two extremes, the neuron assumes an S-shape, as shown in Figure 3-11.

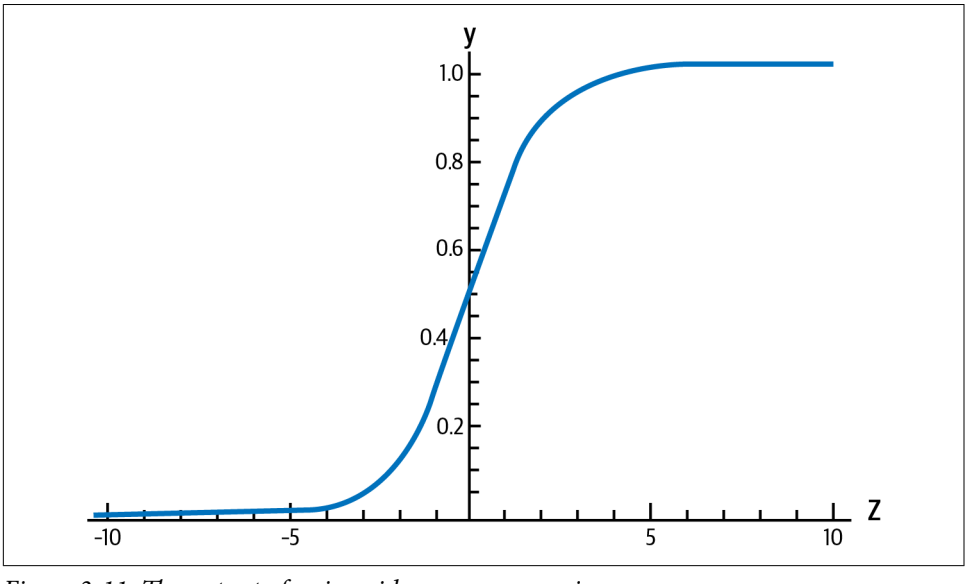


Figure 3-11. The output of a sigmoid neuron as z varies

Tanh neurons use a similar kind of S-shaped nonlinearity, but instead of ranging from 0 to 1, the output of tanh neurons ranges from −1 to 1. As you would expect, they use $f(z) = \tanh(z)$. The resulting relationship between the output y and the logit z is depicted in Figure 3-12. When S-shaped nonlinearities are used, the tanh neuron is often preferred over the sigmoid neuron because it is zero-centered.

A different kind of nonlinearity is used by the *Rectified Linear Unit (ReLU) neuron*. It uses the function $f(z) = \max(0, z)$, resulting in a characteristic hockey-stick-shaped response, as shown in Figure 3-13.

The ReLU has recently become the neuron of choice for many tasks (especially in computer vision) for a number of reasons, despite some drawbacks.[8] We'll discuss these reasons in Chapter 7, as well as strategies to combat the potential pitfalls.

8 Nair, Vinod, and Geoffrey E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines." *Proceedings of the 27th International Conference on Machine Learning* (ICML-10), 2010.

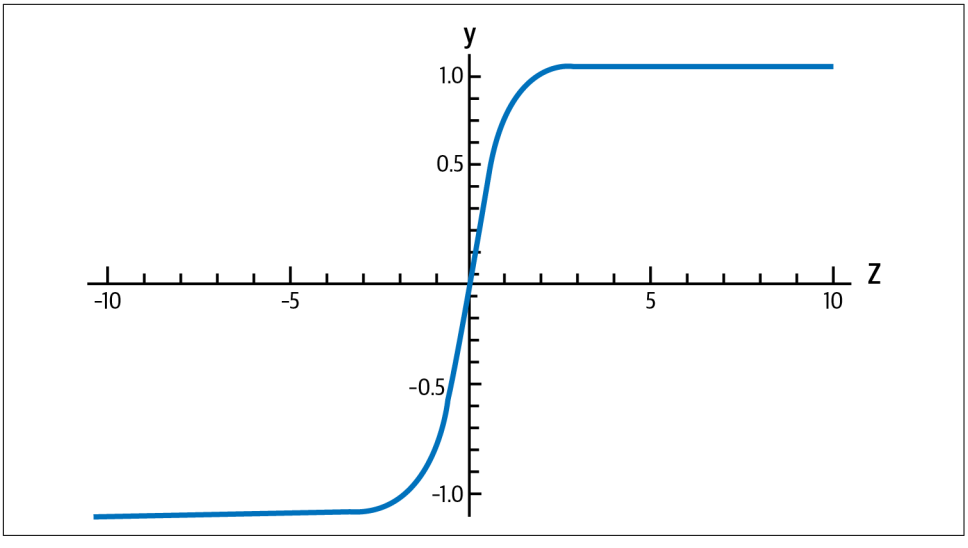


Figure 3-12. The output of a tanh neuron as z varies

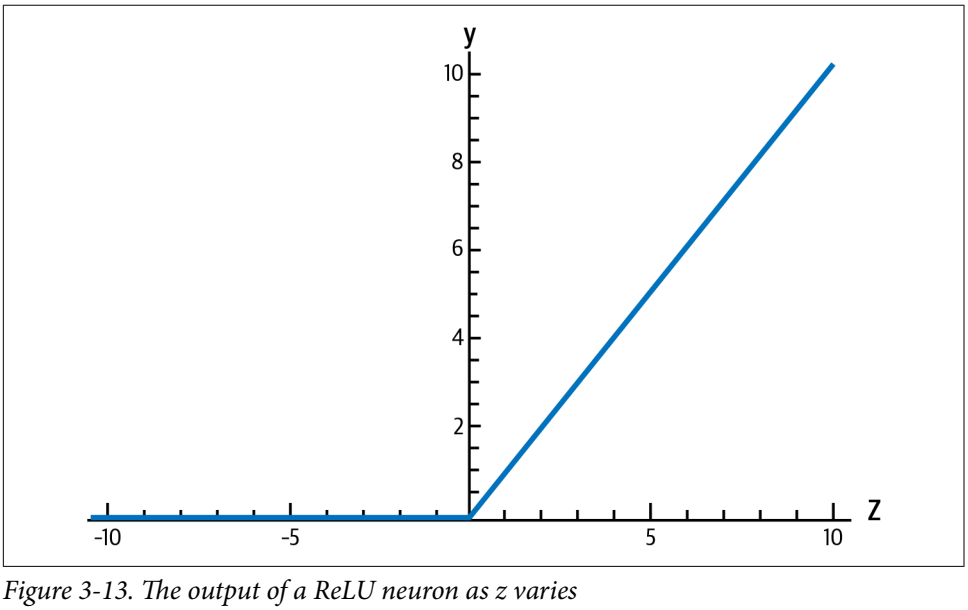


Figure 3-13. The output of a ReLU neuron as z varies

Softmax Output Layers

Oftentimes, we want our output vector to be a probability distribution over a set of mutually exclusive labels. For example, let's say we want to build a neural network to recognize handwritten digits from the MNIST dataset. Each label (0 through 9) is mutually exclusive, but it's unlikely that we will be able to recognize digits with 100% confidence. Using a probability distribution gives us a better idea of how confident we are in our predictions. As a result, the desired output vector is of the following form, where $\sum_{i=0}^{9} p_i = 1$:

$$\begin{bmatrix} p_0 & p_1 & p_2 & p_3 & \cdots & p_9 \end{bmatrix}$$

This is achieved by using a special output layer called a *softmax layer*. Unlike in other kinds of layers, the output of a neuron in a softmax layer depends on the outputs of all the other neurons in its layer. This is because we require the sum of all the outputs to be equal to 1. Letting z_i be the logit of the ith softmax neuron, we can achieve this normalization by setting its output to:

$$y_i = \frac{e^{z}i}{\Sigma_j e^{z}j}$$

A strong prediction would have a single entry in the vector close to 1, while the remaining entries would be close to 0. A weak prediction would have multiple possible labels that are more or less equally likely.

Summary

In this chapter, we've built a basic intuition for machine learning and neural networks. We've talked about the basic structure of a neuron, how feed-forward neural networks work, and the importance of nonlinearity in tackling complex learning problems. In the next chapter, we will begin to build the mathematical background necessary to train a neural network to solve problems. Specifically, we will talk about finding optimal parameter vectors, best practices while training neural networks, and major challenges. In later chapters, we will take these foundational ideas to build more specialized neural architectures.

CHAPTER 4

Training Feed-Forward Neural Networks

The Fast-Food Problem

We're beginning to understand how we can tackle some interesting problems using deep learning, but one big question still remains: how exactly do we figure out what the parameter vectors (the weights for all of the connections in our neural network) should be? This is accomplished by a process commonly referred to as *training* (see Figure 4-1). During training, we show the neural net a large number of training examples and iteratively modify the weights to minimize the errors we make on the training examples. After enough examples, we expect that our neural network will be quite effective at solving the task it's been trained to do.

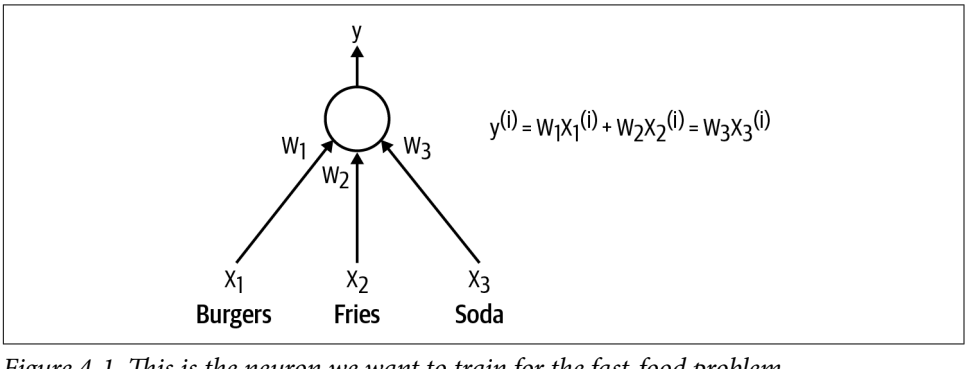


Figure 4-1. This is the neuron we want to train for the fast-food problem

Let's continue with an example from Chapter 3 involving a linear neuron: every single day, we purchase a restaurant meal consisting of burgers, fries, and sodas. We buy some number of servings for each item. We want to be able to predict how much a meal is going to cost us, but the items don't have price tags. The only thing the cashier

will tell us is the total price of the meal. We want to train a single linear neuron to solve this problem. How do we do it?

One idea is to be intelligent about picking our training cases. For one meal we could buy only a single serving of burgers, for another we could buy only a single serving of fries, and then for our last meal we could buy a single serving of soda. In general, intelligently selecting training examples is a good idea. Lots of research shows that by engineering a clever training set, you can make your neural network a lot more effective. The issue with using this approach alone is that in real situations, it rarely ever gets you close to the solution. For example, there's no clear analog of this strategy in image recognition. It's just not a practical solution.

Instead, we try to motivate a solution that works well in general. Let's say we have a large set of training examples. Then we can calculate what the neural network will output on the ith training example using the simple formula in the diagram. We want to train the neuron so that we pick the most optimal weights possible—the weights that minimize the errors we make on the training examples. In this case, let's say we want to minimize the square error over all of the training examples that we encounter. More formally, if we know that $t^{(i)}$ is the true answer for the ith training example, and $y^{(i)}$ is the value computed by the neural network, we want to minimize the value of the error function *E*:

$$E = \frac{1}{2}\Sigma_i\left(t^{(i)} - y^{(i)}\right)^2$$

The squared error is zero when our model makes a perfectly correct prediction on every training example. Moreover, the closer *E* is to 0, the better our model is. As a result, our goal is to select our parameter vector θ (the values for all the weights in our model) such that *E* is as close to 0 as possible.

Now at this point you might be wondering why we need to bother ourselves with error functions when we can treat this problem as a system of equations. After all, we have a bunch of unknowns (weights) and we have a set of equations (one for each training example). That would automatically give us an error of 0, assuming that we have a consistent set of training examples.

That's a smart observation, but the insight unfortunately doesn't generalize well. Remember that although we're using a linear neuron here, linear neurons aren't used very much in practice because they're constrained in what they can learn. And the moment we start using nonlinear neurons like the sigmoidal, tanh, or ReLU neurons we talked about at the end of Chapter 3, we can no longer set up a system of linear equations. Clearly, we need a better strategy to tackle the training process.

Gradient Descent

Let's visualize how we might minimize the squared error over all of the training examples by simplifying the problem. Say our linear neuron has only two inputs (and thus only two weights, w_1 and w_2). Then we can imagine a 3D space where the horizontal dimensions correspond to the weights w_1 and w_2, and the vertical dimension corresponds to the value of the error function *E*. In this space, points in the horizontal plane correspond to different settings of the weights, and the height at those points corresponds to the incurred error. If we consider the errors we make over all possible weights, we get a surface in this 3D space, in particular, a quadratic bowl as shown in Figure 4-2.

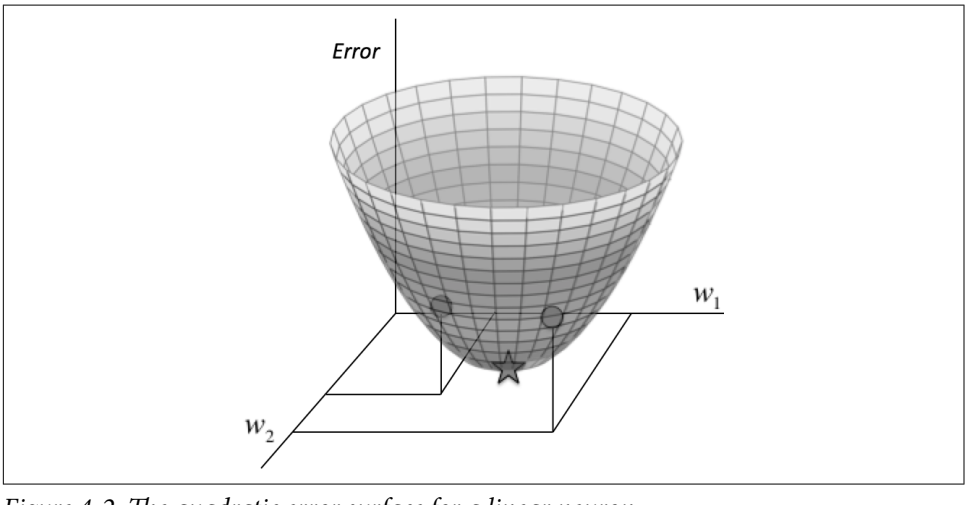


Figure 4-2. The quadratic error surface for a linear neuron

We can also conveniently visualize this surface as a set of elliptical contours, where the minimum error is at the center of the ellipses. In this setup, we are working in a 2D plane where the dimensions correspond to the two weights. Contours correspond to settings of w_1 and w_2 that evaluate to the same value of *E*. The closer the contours are to each other, the steeper the slope. In fact, it turns out that the direction of the steepest descent is always perpendicular to the contours. This direction is expressed as a vector known as the *gradient*.

Now we can develop a high-level strategy for how to find the values of the weights that minimizes the error function. Suppose we randomly initialize the weights of our network so we find ourselves somewhere on the horizontal plane. By evaluating the gradient at our current position, we can find the direction of steepest descent, and we can take a step in that direction. Then we'll find ourselves at a new position that's closer to the minimum than we were before. We can reevaluate the direction of steepest descent by taking the gradient at this new position and taking a step in this

new direction. It's easy to see that, as shown in Figure 4-3, following this strategy will eventually get us to the point of minimum error. This algorithm is known as *gradient descent*, and we'll use it to tackle the problem of training individual neurons and the more general challenge of training entire networks.[1]

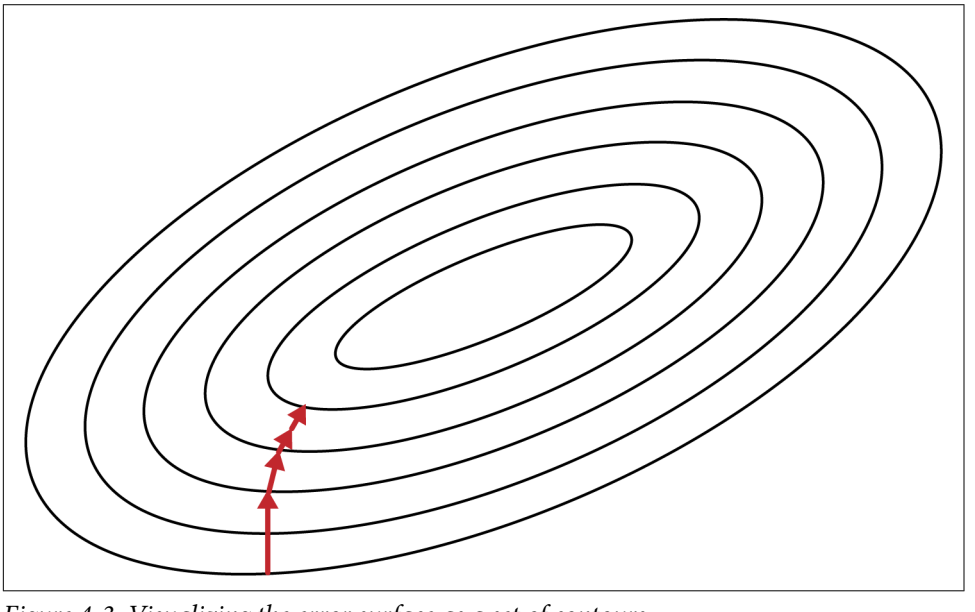


Figure 4-3. Visualizing the error surface as a set of contours

The Delta Rule and Learning Rates

Before we derive the exact algorithm for training our fast-food neuron, we have a quick note on *hyperparameters*. In addition to the weight parameters defined in our neural network, learning algorithms also require a couple of additional parameters to carry out the training process. One of these so-called hyperparameters is the *learning rate*.

In practice, at each step of moving perpendicular to the contour, we need to determine how far we want to walk before recalculating our new direction. This distance needs to depend on the steepness of the surface. Why? The closer we are to the minimum, the shorter we want to step forward. We know we are close to the minimum because the surface is a lot flatter, so we can use the steepness as an indicator of how close we are to the minimum. However, if our error surface is rather mellow, training can potentially take a large amount of time. As a result, we often multiply the

1 Rosenbloom, P. "The Method of Steepest Descent." *Proceedings of Symposia in Applied Mathematics*. Vol. 6. 1956.

gradient by a factor ϵ, the learning rate. Picking the learning rate is a hard problem (Figure 4-4). As we just discussed, if we pick a learning rate that's too small, we risk taking too long during the training process. But if we pick a learning rate that's too big, we'll mostly likely start diverging away from the minimum. In Chapter 5, we'll learn about various optimization techniques that utilize adaptive learning rates to automate the process of selecting learning rates.

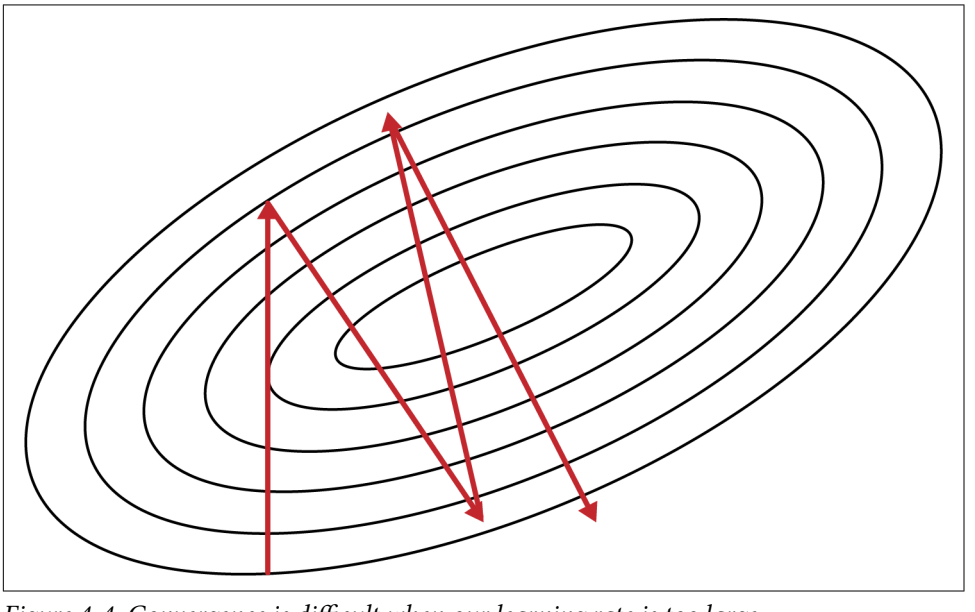


Figure 4-4. Convergence is difficult when our learning rate is too large

Now, we are finally ready to derive the *delta rule* for training our linear neuron. In order to calculate how to change each weight, we evaluate the gradient, which is essentially the partial derivative of the error function with respect to each of the weights. In other words, we want:

$$\Delta w_k = -\epsilon \frac{\partial E}{\partial w_k}$$

$$= -\epsilon \frac{\partial}{\partial w_k}\left(\frac{1}{2}\Sigma_i\left(t^{(i)} - y^{(i)}\right)^2\right)$$

$$= \Sigma_i \epsilon\left(t^{(i)} - y^{(i)}\right)\frac{\partial y_i}{\partial w_k}$$

$$= \Sigma_i \epsilon x_k^{(i)}\left(t^{(i)} - y^{(i)}\right)$$

Applying this method of changing the weights at every iteration, we are finally able to utilize gradient descent.

Gradient Descent with Sigmoidal Neurons

In this section and the next, we will deal with training neurons and neural networks that utilize nonlinearities. We use the sigmoidal neuron as a model, and leave the derivations for other nonlinear neurons as an exercise for you. For simplicity, we assume that the neurons do not use a bias term, although our analysis easily extends to this case. We merely need to assume that the bias is a weight on an incoming connection whose input value is always one.

Let's recall the mechanism by which logistic neurons compute their output value from their inputs:

$$z = \Sigma_k w_k x_k$$

$$y = \frac{1}{1 + e^{-z}}$$

The neuron computes the weighted sum of its inputs, the logit z. It then feeds its logit into the input function to compute y, its final output. Fortunately for us, these functions have nice derivatives, which makes learning easy! For learning, we want to compute the gradient of the error function with respect to the weights. To do so, we start by taking the derivative of the logit with respect to the inputs and the weights:

$$\frac{\partial z}{\partial w_k} = x_k$$

$$\frac{\partial z}{\partial x_k} = w_k$$

Also, quite surprisingly, the derivative of the output with respect to the logit is quite simple if you express it in terms of the output:

$$\begin{aligned}
\frac{dy}{dz} &= \frac{e^{-z}}{\left(1 + e^{-z}\right)^2} \\
&= \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}} \\
&= \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}}\right) \\
&= y(1 - y)
\end{aligned}$$

We then use the chain rule to get the derivative of the output with respect to each weight:

$$\frac{\partial y}{\partial w_k} = \frac{dy}{dz}\frac{\partial z}{\partial w_k} = x_k y(1-y)$$

Putting all of this together, we can now compute the derivative of the error function with respect to each weight:

$$\frac{\partial E}{\partial w_k} = \Sigma_i \frac{\partial E}{\partial y^{(i)}}\frac{\partial y^{(i)}}{\partial w_k} = -\Sigma_i x_k^{(i)} y^{(i)}\left(1-y^{(i)}\right)\left(t^{(i)}-y^{(i)}\right)$$

Thus, the final rule for modifying the weights becomes:

$$\Delta w_k = \Sigma_i \epsilon x_k^{(i)} y^{(i)}\left(1-y^{(i)}\right)\left(t^{(i)}-y^{(i)}\right)$$

As you may notice, the new modification rule is just like the delta rule, except with extra multiplicative terms included to account for the logistic component of the sigmoidal neuron.

The Backpropagation Algorithm

Now we're finally ready to tackle the problem of training multilayer neural networks (instead of just single neurons). To accomplish this task, we'll use an approach known as *backpropagation*, pioneered by David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams in 1986.[2] So what's the idea behind backpropagation? We don't know what the hidden units ought to be doing, but what we can do is compute how fast the error changes as we change a hidden activity. From there, we can figure out how fast the error changes when we change the weight of an individual connection. Essentially, we'll be trying to find the path of steepest descent. The only catch is that we're going to be working in an extremely high-dimensional space. We start by calculating the error derivatives with respect to a single training example.

Each hidden unit can affect many output units. Thus, we'll have to combine many separate effects on the error in an informative way. Our strategy will be one of dynamic programming. Once we have the error derivatives for one layer of hidden units, we'll use them to compute the error derivatives for the activities of the layer below. And once we find the error derivatives for the activities of the hidden units, it's

2 Rumelhart, David E., Geoffrey E. Hinton, and Ronald J. Williams. "Learning Representations by Back-Propagating Errors." *Cognitive Modeling* 5.3 (1988): 1.

quite easy to get the error derivatives for the weights leading into a hidden unit. We'll redefine some notation for ease of discussion and refer to Figure 4-5.

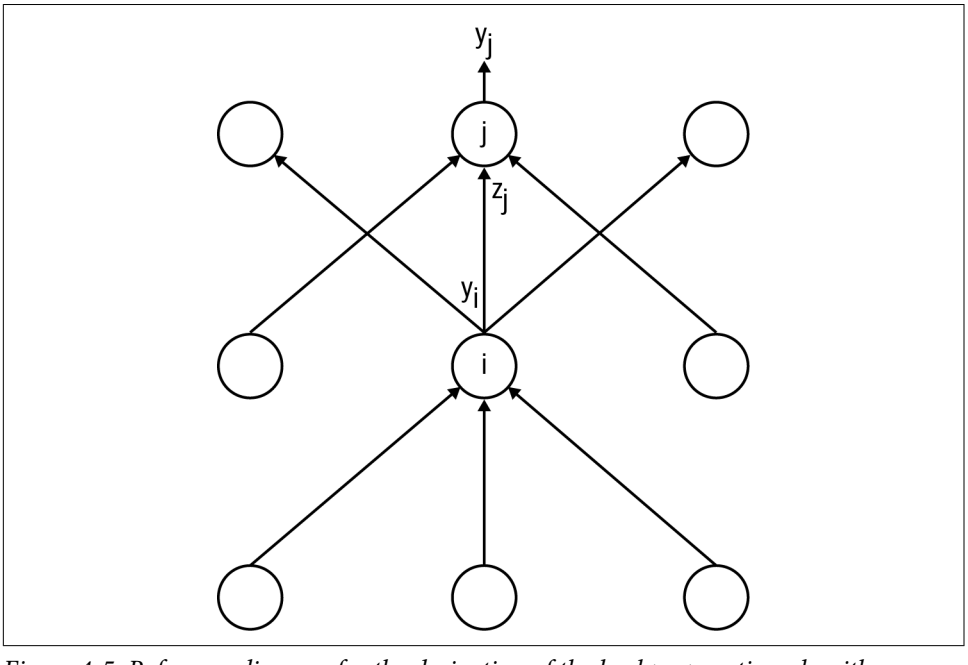


Figure 4-5. Reference diagram for the derivation of the backpropagation algorithm

The subscript we use will refer to the layer of the neuron. The symbol y will refer to the activity of a neuron, as usual. Similarly, the symbol z will refer to the logit of the neuron. We start by taking a look at the base case of the dynamic programming problem. Specifically, we calculate the error function derivatives at the output layer:

$$E = \frac{1}{2}\Sigma_{j \in output}\left(t_j - y_j\right)^2 \Rightarrow \frac{\partial E}{\partial y_j} = -\left(t_j - y_j\right)$$

Now we tackle the inductive step. Let's presume we have the error derivatives for layer j. We next aim to calculate the error derivatives for the layer below it, layer i. To do so, we must accumulate information about how the output of a neuron in layer i affects the logits of every neuron in layer j. This can be done as follows, using the fact that the partial derivative of the logit with respect to the incoming output data from the layer beneath is merely the weight of the connection w_{ij}:

$$\frac{\partial E}{\partial y_i} = \Sigma_j \frac{\partial E}{\partial z_j}\frac{dz_j}{dy_i} = \Sigma_j w_{ij}\frac{\partial E}{\partial z_j}$$

Furthermore, we observe the following:

$$\frac{\partial E}{\partial z_j} = \frac{\partial E}{\partial y_j}\frac{dy_j}{dz_j} = y_j\left(1 - y_j\right)\frac{\partial E}{\partial y_j}$$

Combining these two, we can finally express the error derivatives of layer i in terms of the error derivatives of layer j:

$$\frac{\partial E}{\partial y_i} = \Sigma_j w_{ij} y_j\left(1 - y_j\right)\frac{\partial E}{\partial y_j}$$

Once we've gone through the whole dynamic programming routine, having filled up the table appropriately with all of our partial derivatives (of the error function with respect to the hidden unit activities), we can then determine how the error changes with respect to the weights. This gives us a way to modify the weights after each training example:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial z_j}{\partial w_{ij}}\frac{\partial E}{\partial z_j} = y_i y_j\left(1 - y_j\right)\frac{\partial E}{\partial y_j}$$

Finally, to complete the algorithm, just as before, we merely sum up the partial derivatives over all the training examples in our dataset. This gives us the following modification formula:

$$\Delta w_{ij} = \ -\Sigma_{k \in dataset}\epsilon y_i^{(k)} y_j^{(k)}\left(1 - y_j^{(k)}\right)\frac{\partial E^{(k)}}{\partial y_j^{(k)}}$$

This completes our description of the backpropagation algorithm.

Stochastic and Minibatch Gradient Descent

In the algorithms we described in "The Backpropagation Algorithm" on page 61, we used a version of gradient descent known as *batch gradient descent*. The idea behind batch gradient descent is that we use our entire dataset to compute the error surface and then follow the gradient to take the path of steepest descent. For a simple quadratic error surface, this works quite well. But in most cases, our error surface may be a lot more complicated. Let's consider the scenario in Figure 4-6.

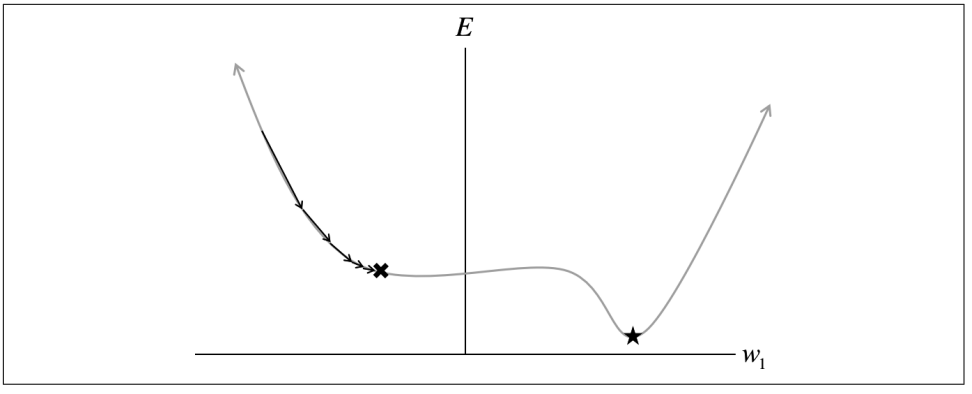


Figure 4-6. Batch gradient descent is sensitive to saddle points, which can lead to premature convergence

We have only a single weight, and we use random initialization and batch gradient descent to find its optimal setting. The error surface, however, has a flat region (also known as saddle point in high-dimensional spaces), and if we get unlucky, we might find ourselves getting stuck while performing gradient descent.

Another potential approach is *stochastic gradient descent* (SGD), where at each iteration, our error surface is estimated with respect to only a single example. This approach is illustrated by Figure 4-7, where instead of a single static error surface, our error surface is dynamic. As a result, descending on this stochastic surface significantly improves our ability to navigate flat regions.

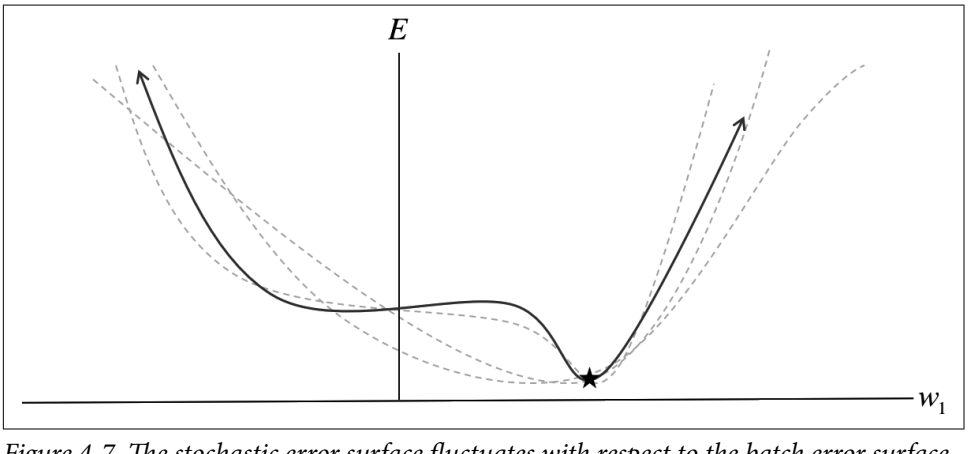


Figure 4-7. The stochastic error surface fluctuates with respect to the batch error surface, enabling saddle point avoidance

The major pitfall of SGD, however, is that looking at the error incurred one example at a time may not be a good enough approximation of the error surface. This, in turn, could potentially make gradient descent take a significant amount of time. One way to combat this problem is using *minibatch gradient descent*. In minibatch gradient descent, at every iteration we compute the error surface with respect to some subset of the total dataset (instead of just a single example). This subset is called a *minibatch*, and in addition to the learning rate, minibatch size is another hyperparameter. Minibatches strike a balance between the efficiency of batch gradient descent and the local-minima avoidance afforded by stochastic gradient descent. In the context of backpropagation, our weight update step becomes:

$$\Delta w_{ij} = -\Sigma_{k \in minibatch} \epsilon y_i^{(k)} y_j^{(k)} \left(1 - y_j^{(k)}\right) \frac{\partial E^{(k)}}{\partial y_j^{(k)}}$$

This is identical to what we derived in the previous section, but instead of summing over all the examples in the dataset, we sum over the examples in the current minibatch. For a more theoretical discussion of why SGD and minibatch gradient descent result in an unbiased estimate of the gradient over the total dataset, please refer to "Neural Net Learning Theory" on page 74.

Test Sets, Validation Sets, and Overfitting

One of the major issues with artificial neural networks is that the models are quite complicated. For example, let's consider a neural network that pulls data from an image from the MNIST database (28 × 28 pixels), feeds into two hidden layers with 30 neurons, and finally reaches a softmax layer of 10 neurons. The total number of parameters in the network is nearly 25,000. This can be quite problematic, and to understand why, let's consider a new toy example, illustrated in Figure 4-8.

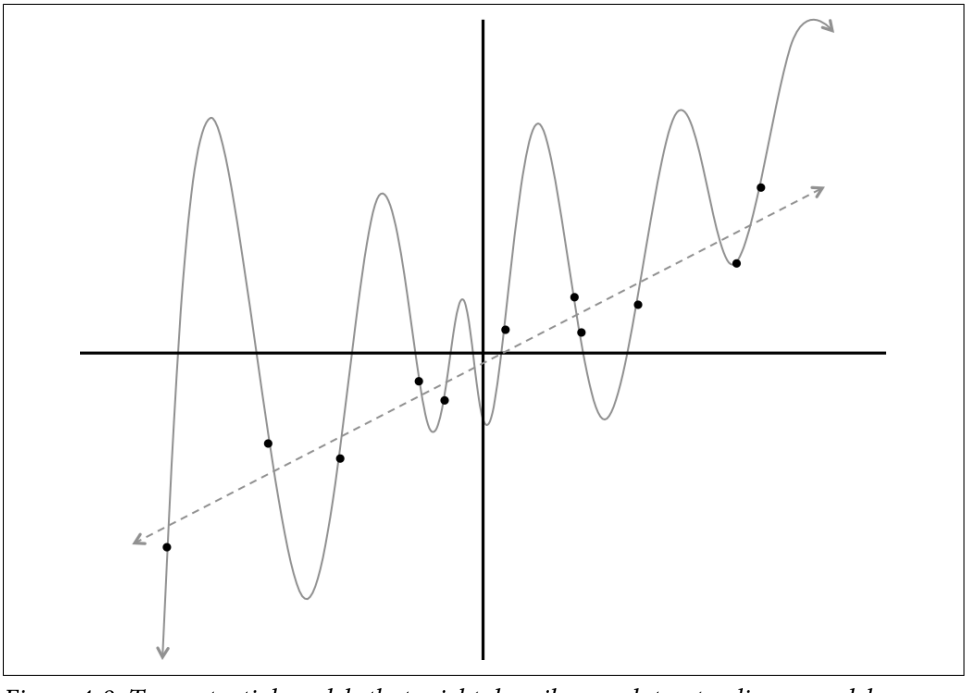


Figure 4-8. Two potential models that might describe our dataset: a linear model versus a degree 12 polynomial

We are given a bunch of data points on a flat plane, and our goal is to find a curve that best describes this dataset (i.e., will allow us to predict the y coordinate of a new point given its x coordinate). Using the data, we train two different models: a linear model and a degree 12 polynomial. Which curve should we trust? The line that gets almost no training example correct? Or the complicated curve that hits every single point in the dataset? At this point we might trust the linear fit because it seems much less contrived. But just to be sure, let's add more data to our dataset. The result is shown in Figure 4-9.

Now the verdict is clear: the linear model is not only better subjectively but also quantitatively (measured using the squared error metric). This leads to an interesting point about training and evaluating machine learning models. By building a very complex model, it's quite easy to perfectly fit our training dataset because we give our model enough degrees of freedom to contort itself to fit the observations in the training set. But when we evaluate such a complex model on new data, it performs poorly. In other words, the model does not *generalize* well. This is a phenomenon called *overfitting*, and it is one of the biggest challenges that a machine learning engineer must combat. This becomes an even more significant issue in deep learning, where our neural networks have large numbers of layers containing many neurons.

The number of connections in these models is astronomical, reaching the millions. As a result, overfitting is commonplace.

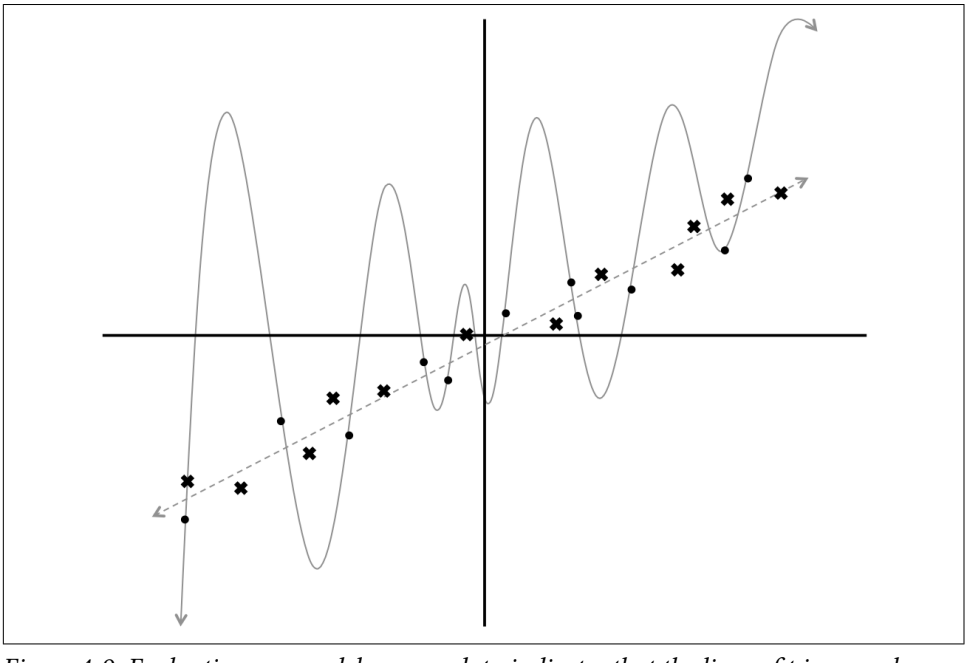


Figure 4-9. Evaluating our model on new data indicates that the linear fit is a much better model than the degree 12 polynomial

Let's see how this looks in the context of a neural network. Say we have a neural network with two inputs, a softmax output of size 2, and a hidden layer with 3, 6, or 20 neurons. We train these networks using minibatch gradient descent (batch size 10), and the results, visualized using ConvNetJS, are shown in Figure 4-10.

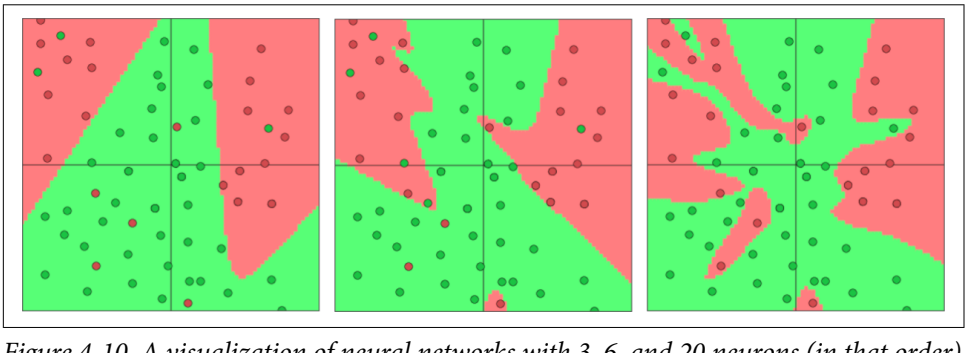


Figure 4-10. A visualization of neural networks with 3, 6, and 20 neurons (in that order) in their hidden layer

It's already quite apparent from these images that as the number of connections in our network increases, so does our propensity to overfit to the data. We can similarly see the phenomenon of overfitting as we make our neural networks deep. These results are shown in Figure 4-11.

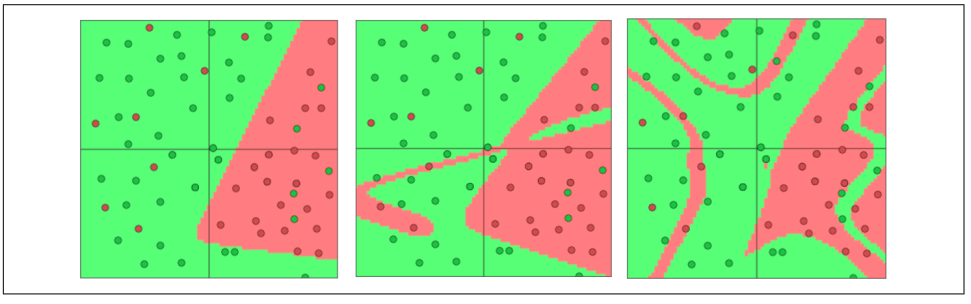


Figure 4-11. Neural networks with one, two, and four hidden layers (in that order) of three neurons each

This leads to three major observations. First, the machine learning engineer is always working with a direct trade-off between overfitting and model complexity. If the model isn't complex enough, it may not be powerful enough to capture all of the useful information necessary to solve a problem. However, if our model is very complex (especially if we have a limited amount of data at our disposal), we run the risk of overfitting. Deep learning takes the approach of solving complex problems with complex models and taking additional countermeasures to prevent overfitting. We'll see a lot of these measures in this and later chapters.

Second, it is misleading to evaluate a model using the data we used to train it. Using the example in Figure 4-8, this would falsely suggest that the degree 12 polynomial model is preferable to a linear fit. As a result, we almost never train our model on the entire dataset. Instead, we split up our data into a *training set* and a *test set* (Figure 4-12). This enables us to make a fair evaluation of our model by directly measuring how well it generalizes on new data it has not yet seen.



In the real world, large datasets are hard to come by, so it might seem like a waste to not use all of the data at our disposal during the training process. Consequently, it may be tempting to reuse training data for testing or cut corners while compiling test data. Be forewarned: if the test set isn't well constructed, we won't be able draw any meaningful conclusions about our model.

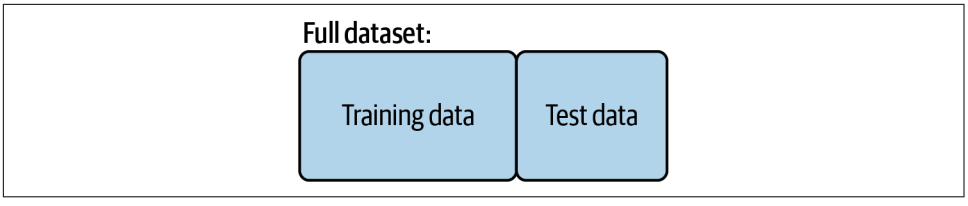


Figure 4-12. Nonoverlapping training and test sets

Third, it's quite likely that while we're training our data, there's a point in time when instead of learning useful features, we start overfitting to the training set. To avoid that, we want to be able to stop the training process as soon as we start overfitting to prevent poor generalization. To do this, we divide our training process into *epochs*. An epoch is a single iteration over the entire training set. If we have a training set of size d and we are doing minibatch gradient descent with batch size b, then an epoch would be equivalent to $\frac{d}{b}$ model updates. At the end of each epoch, we want to measure how well our model is generalizing. To do this, we use an additional *validation set*, which is shown in Figure 4-13.

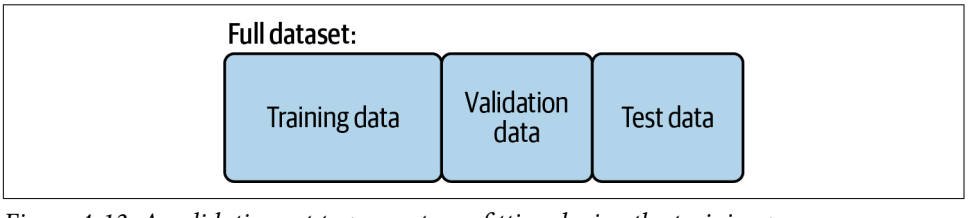


Figure 4-13. A validation set to prevent overfitting during the training process

At the end of an epoch, the validation set will tell us how the model does on data it has yet to see. If the accuracy on the training set continues to increase while the accuracy on the validation set stays the same (or decreases), it's a good sign that it's time to stop training because we're overfitting.

The validation set is also helpful as a proxy measure of accuracy during the process of *hyperparameter optimization*. We've covered several hyperparameters so far (learning rate, minibatch size, etc.), but we have yet to develop a framework for how to find the optimal values for these hyperparameters. One potential way to find the optimal setting of hyperparameters is by applying a *grid search,* where we pick a value for each hyperparameter from a finite set of options (e.g., $\epsilon \in \{0.001, 0.01, 0.1\}$, batch size $\in \{16, 64, 128\}, \ldots$), and train the model with every possible permutation of hyperparameter choices. We elect the combination

of hyperparameters with the best performance on the validation set and report the accuracy of the model trained with the best combination on the test set.[3]

With this in mind, before we jump into describing the various ways to directly combat overfitting, let's outline the workflow we use when building and training deep learning models. The workflow is described in detail in Figure 4-14. It is a tad intricate, but it's critical to understand the pipeline to ensure that we're properly training our neural networks.

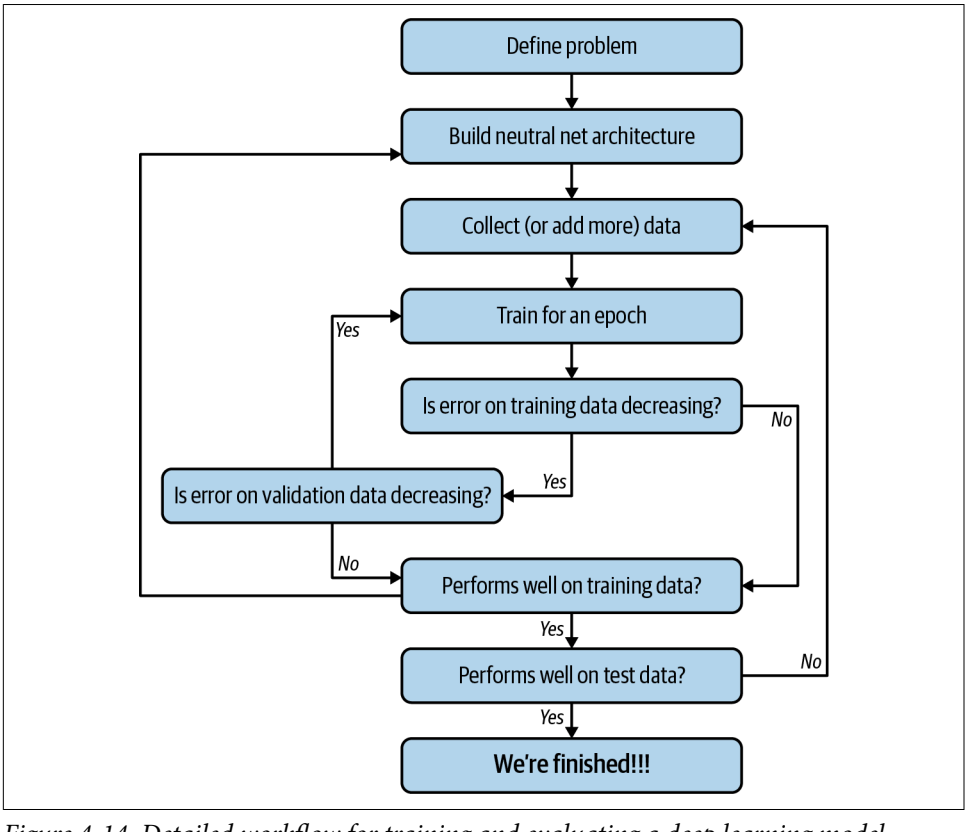


Figure 4-14. Detailed workflow for training and evaluating a deep learning model

First, we define our problem rigorously. This involves determining our inputs, the potential outputs, and the vectorized representations of both. For instance, let's say our goal was to train a deep learning model to identify cancer. Our input would be an RBG image, which can be represented as a vector of pixel values. Our output would be a probability distribution over three mutually exclusive possibilities: (1) normal,

3 Nelder, John A., and Roger Mead. "A Simplex Method for Function Minimization." *The Computer Journal* 7.4 (1965): 308-313.

(2) benign tumor (a cancer that has yet to metastasize), or (3) malignant tumor (a cancer that has already metastasized to other organs).

After we define our problem, we need to build a neural network architecture to solve it. Our input layer would have to be of appropriate size to accept the raw data from the image, and our output layer would have to be a softmax of size 3. We will also have to define the internal architecture of the network (number of hidden layers, the connectivities, etc.). We'll further discuss the architecture of image recognition models when we talk about convolutional neural networks in Chapter 6. At this point, we also want to collect a significant amount of data for training or modeling. This data would probably be in the form of uniformly sized pathological images that have been labeled by a medical expert. We shuffle and divide this data up into separate training, validation, and test sets.

Finally, we're ready to begin gradient descent. We train the model on our training set for an epoch at a time. At the end of each epoch, we ensure that our error on the training set and validation set is decreasing. When one of these stops improving, we terminate and make sure we're happy with the model's performance on the test data. If we're unsatisfied, we need to rethink our architecture or reconsider whether the data we collect has the information required to make the prediction we're interested in making. If our training set error stopped improving, we probably need to do a better job of capturing the important features in our data. If our validation set error stopped improving, we probably need to take measures to prevent overfitting.

If, however, we are happy with the performance of our model on the training data, then we can measure its performance on the test data, which the model has never seen before this point. If it is unsatisfactory, we need more data in our dataset because the test set seems to consist of example types that weren't well represented in the training set. Otherwise, we are finished!

Preventing Overfitting in Deep Neural Networks

Several techniques have been proposed to prevent overfitting during the training process. In this section, we'll discuss these techniques in detail.

One method of combatting overfitting is called *regularization*. Regularization modifies the objective function that we minimize by adding additional terms that penalize large weights. We change the objective function so that it becomes $Error + \lambda f(\theta)$, where $f(\theta)$ grows larger as the components of θ grow larger, and λ is the regularization strength (another hyperparameter). The value we choose for λ determines how much we want to protect against overfitting. A $\lambda = 0$ implies that we do not take any measures against the possibility of overfitting. If λ is too large, then our model will prioritize keeping θ as small as possible over trying to find the parameter values that

perform well on our training set. As a result, choosing λ is a very important task and can require some trial and error.

The most common type of regularization in machine learning is *L2 regularization*.[4] It can be implemented by augmenting the error function with the squared magnitude of all weights in the neural network. In other words, for every weight w in the neural network, we add $\frac{1}{2}\lambda w^2$ to the error function. The L2 regularization has the intuitive interpretation of heavily penalizing peaky weight vectors and preferring diffuse weight vectors. This has the appealing property of encouraging the network to use all of its inputs a little rather than using only some of its inputs a lot. Of particular note is that during the gradient descent update, using the L2 regularization ultimately means that every weight is decayed linearly to zero. Because of this phenomenon, L2 regularization is also commonly referred to as *weight decay*.

We can visualize the effects of L2 regularization using ConvNetJS. Similar to Figures 2-10 and 2-11, we use a neural network with 2 inputs, a softmax output of size 2, and a hidden layer with 20 neurons. We train the networks using minibatch gradient descent (batch size 10) and regularization strengths of 0.01, 0.1, and 1. The results can be seen in Figure 4-15.

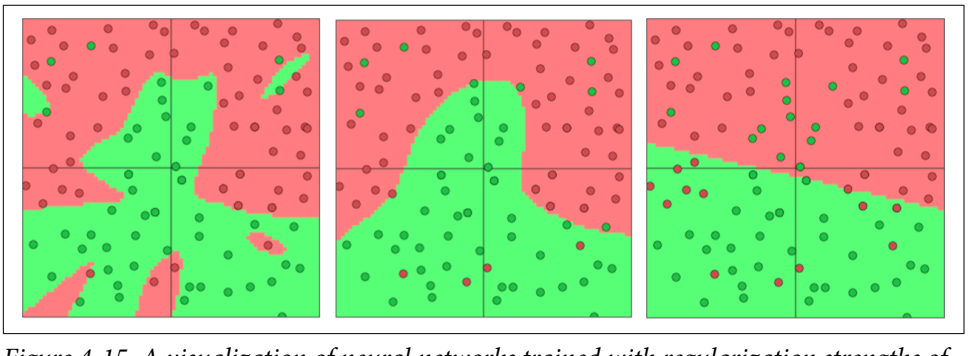


Figure 4-15. A visualization of neural networks trained with regularization strengths of 0.01, 0.1, and 1 (in that order)

Another common type of regularization is *L1 regularization*. Here, we add the term $\lambda|w|$ for every weight w in the neural network. The L1 regularization has the intriguing property that it leads the weight vectors to become sparse during optimization (i.e., close to exactly zero). Neurons with L1 regularization end up using only a small subset of their most important inputs and become quite resistant to noise in the inputs. In comparison, weight vectors from L2 regularization are usually diffuse, small numbers. L1 regularization is useful when you want to understand exactly

4 Tikhonov, Andrei Nikolaevich, and Vladlen Borisovich Glasko. "Use of the Regularization Method in Non-Linear Problems." *USSR Computational Mathematics and Mathematical Physics* 5.3 (1965): 93-107.

which features are contributing to a decision. If this level of feature analysis isn't necessary, we prefer to use L2 regularization because it empirically performs better.

Max norm constraints have a similar goal of attempting to restrict θ from becoming too large, but they do this more directly.[5] Max norm constraints enforce an absolute upper bound on the magnitude of the incoming weight vector for every neuron and use projected gradient descent to enforce the constraint. So any time a gradient descent step moves the incoming weight vector such that $||w||_2 > c$, we project the vector back onto the ball (centered at the origin) with radius c. Typical values of c are 3 and 4. One of the nice properties is that the parameter vector cannot grow out of control (even if the learning rates are too high) because the updates to the weights are always bounded.

Dropout is a different kind of method for preventing overfitting that has become one of the most favored methods of preventing overfitting in deep neural networks.[6] While training, dropout is implemented by only keeping a neuron active with some probability p (a hyperparameter), or setting it to zero otherwise. Intuitively, this forces the network to be accurate even in the absence of certain information. It prevents the network from becoming too dependent on any one neuron (or any small combination of neurons). Expressed more mathematically, it prevents overfitting by providing a way of approximately combining exponentially many different neural network architectures efficiently. The process of dropout is expressed in Figure 4-16.

Dropout is pretty intuitive, but there are some important intricacies to consider. First, we'd like the outputs of neurons during test time to be equivalent to their expected outputs at training time. We could fix this naively by scaling the output at test time. For example, if $p = 0.5$, neurons must halve their outputs at test time in order to have the same (expected) output they would have during training. This is easy to see because a neuron's output is set to 0 with probability $1 - p$. This means that if a neuron's output prior to dropout was x, then after dropout, the expected output would be $E[\text{output}] = px + (1 - p) \cdot 0 = px$. This naive implementation of dropout is undesirable, however, because it requires scaling of neuron outputs at test time. Test-time performance is extremely critical to model evaluation, so it's always preferable to use *inverted dropout*, where the scaling occurs at training time instead of at test time. In inverted dropout, any neuron whose activation hasn't been silenced has its output divided by p before the value is propagated to the next layer. With this fix, $E[\text{output}] = p \cdot \frac{x}{p} + (1 - p) \cdot 0 = x$, and we can avoid arbitrarily scaling neuronal output at test time.

5 Srebro, Nathan, Jason DM Rennie, and Tommi S. Jaakkola. "Maximum-Margin Matrix Factorization." *NIPS*, Vol. 17, 2004.

6 Srivastava, Nitish, et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15.1 (2014): 1929-1958.

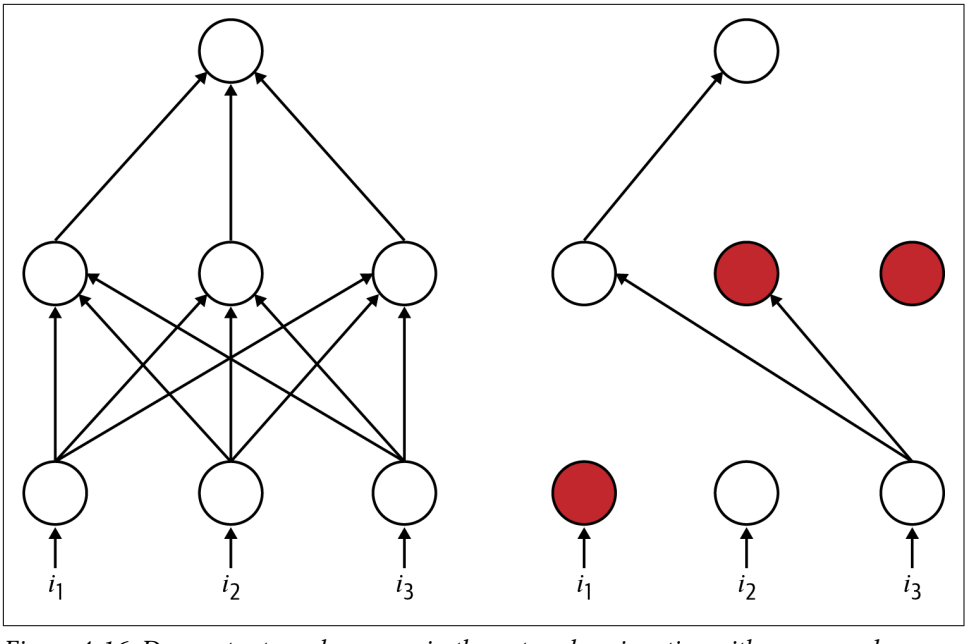


Figure 4-16. Dropout sets each neuron in the network as inactive with some random probability during each minibatch of training

Neural Net Learning Theory

Let's cover some of the theory underpinning SGD and minibatch gradient descent. We noted some of the empirical gains from SGD and minibatch gradient descent in this chapter. Here, we try to understand why it is even "OK," in theory, to use these learning algorithms as a substitute for the gradient update over the entire dataset, which can often be intractable with many training examples. The intractability arises from having to take the partial derivative with respect to every weight in the neural network for every training example in any given iteration, which is prohibitive for large datasets and neural networks of even moderate size.

Here, for simplicity, we will refer to X as the *data matrix*, which is a matrix of dimension n by d, where n refers to the number of inputs in the entire dataset and d refers to the number of features associated with any given input. In other words, each row of X corresponds to a single training example. In addition to X, we have y, a vector of labels, or desired outputs, associated with each input of the data matrix. As can be inferred, y is of dimension n by 1. We refer to $f(\cdot, \theta)$ as the function defined by the neural network, and θ as the weights (which we intend to learn through the training process) that parametrize it. Finally, we refer to $L(\widehat{y}, y)$ as the error, or loss, function when comparing the predicted output $\widehat{y}$ with the true output y.

Let's start with some basic assumptions. Where do X and y even come from? We assume that there is some underlying data-generating process from which X and y are produced. That is, there exists a joint distribution over inputs and labels $p(x, y)$ from which the dataset we observe has been randomly sampled. In classical learning theory, we would like our learned function, or neural network $f(\cdot, \theta)$, to minimize the objective:

$$E_{p(x, y)}[L(f(x, \theta), y)]$$

or the expected loss with respect to the true distribution $p(x, y)$. We term this objective the *population risk*. Why minimize the population risk? Intuitively, the more popular any given pairing of input datapoint $x^{(i)}$ and output label $y^{(i)}$ under the true distribution, the more we would like weight the output of $f\left(x^{(i)}, \theta\right)$ resembling $y^{(i)}$. $L\left(f\left(x^{(i)}, \theta\right), y^{(i)}\right)$ quantifies how well (or in this case, equivalently, how poorly given L is an error function) the output of $f\left(x^{(i)}, \theta\right)$ resembles $y^{(i)}$. The optimal weight is simply $p\left(x^{(i)}, y^{(i)}\right)$, which quantifies the popularity of the pairing. Putting this logic together over all possible pairings, we arrive at the population risk objective. Note that this logic holds regardless of whether *x* and *y* are discrete or continuous.

Unfortunately, we have no access to the true distribution. If we did, our problem would already be solved (more accurately, is instantly solved when *y* is discrete). The best we can do is approximate $p(x, y)$ with the data we see. We call this empirical distribution $p_D(x, y)$, which, in the limit of infinite data, converges to $p(x, y)$, making any gradient update based on the empirical distribution unbiased. The domain of the empirical distribution is our training set. Instead of minimizing the population risk, we minimize the *empirical risk* as a proxy:

$$E_{p_D(x, y)}[L(f(x; \theta), y)] = \frac{1}{n}\Sigma_{i=1}^{n} L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right)$$

Now, we would like to find a set of weights θ, that minimize the empirical risk. We do that by taking the gradient of the objective with respect to θ:

$$\begin{aligned}
&\nabla_\theta E_{p_D(x, y)}[L(f(x; \theta), y)] \\
&= \nabla_\theta \frac{1}{n}\Sigma_{i=1}^{n} L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right) \\
&= \frac{1}{n}\Sigma_{i=1}^{n} \nabla_\theta L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right)
\end{aligned}$$

We now see that the newly introduced empirical risk minimization is the theoretical motivation for the gradient update over the entire dataset presented in "The Backpropagation Algorithm" on page 61. The only, subtle difference is the factor $\frac{1}{n}$, which is a constant that can be rolled into the learning rate. Extending the last equality, we have:

$$\frac{1}{n}\sum_{i=1}^{n} \nabla_\theta L\left(f\left(x^{(i)}; \theta\right), y^{(i)}\right) = E_{p_D(x,y)}[\nabla_\theta L(f(x; \theta), y)]$$

We can do something that uses almost the exact same logic, which allows us to use the empirical risk as a proxy for population risk: approximate the expectation with respect to the empirical distribution via sampling. Again, this is unbiased since we achieve the empirical distribution in the limit of infinite samples. The number of samples is left as a hyperparameter, where using a single sample has been popularly termed SGD; using a relatively small number of samples has been popularly termed minibatch gradient descent.

Summary

In this chapter, we've learned all of the basics involved in training feed-forward neural networks. We've talked about gradient descent, the backpropagation algorithm, as well as various methods we can use to prevent overfitting. In the next chapter, we'll put these lessons into practice when we use the PyTorch library to efficiently implement our first neural networks. Then in Chapter 6, we'll return to the problem of optimizing objective functions for training neural networks and design algorithms to significantly improve performance. These improvements will enable us to process much more data, which means we'll be able to build more comprehensive models.